

SoK: Pseudorandom Generation for Masked Cryptographic Implementation

Rei Ueno
Kyoto University
Kyoto, Japan
ueno.rei.2e@kyoto-u.ac.jp

Naofumi Homma
Tohoku University
Sendai, Japan
naofumi.homma.c8@tohoku.ac.jp

Kazuhiko Minematsu
NEC Corporation
Kawasaki, Japan
k-minematsu@nec.com

Abstract—This paper investigates pseudorandom generation in the context of masked cryptographic implementation. Although masking and pseudorandom generators (PRGs) have been distinctly studied for a long time, there has been little study on how randomness should be generated in masked implementation. The lack of analysis on mask-bits generators obscures the practical security of masked cryptographic implementation, potentially confusing practitioners (*e.g.*, designer, implementer, and evaluator) about how to realize them securely. This paper provides a novel viewpoint and comprehensive analyses by developing three new models, each of which corresponds to a practical scenario: leakage assessment, quantitative evaluation of side-channel security (*e.g.*, success rate), and practical deployment. We reveal what PRG properties are required. In particular, we support a long-held belief/folklore through proof and experiment: *for the output of PRG for masking, cryptographic security (i.e., unpredictability) is sufficient but not necessary; statistical uniformity alone is necessary*. In addition, we thoroughly investigate the SCA security of PRGs in the wild within the masking context. We conclude this paper with some recommendations for practitioners and a proposal of leakage-resilient method offering comparative performance.

1. Introduction

1.1. Background

Masking is a promising countermeasure against side-channel attacks (SCAs) on cryptographic implementation [28]. Since the introduction of major SCA—differential power analysis (DPA) by Kocher *et al.* [87]—both the theoretical and practical aspects of masking have been studied extensively. Assuming that masking is correctly implemented, its formal security has been proven; that is, a polynomial increase of masking order yields an exponential increase of the number of traces for a successful attack [12], [28], [77], [98]. Additionally, numerous tools/methodologies for leakage detection, design automation of masked implementation, and masked open-source implementations have been developed (*e.g.*, [9], [14], [83], [84], [111], [116], [117], [126], [127]), which make sound masked implementations available in practice. Meanwhile, masked implementation should carefully account for physical defaults—glitch,

coupling, and microarchitectural leakage—that may degrade the effective security order due to an unintended cross-share interaction [6], [43], [44], [61], [63], [93], [103], [110]; however, some masking schemes (*e.g.*, threshold implementation [64], [104]), masked implementations, and tools addressed this issue [95], [112], [116], [117], [133], [134].

Masked cryptographic implementations always require randomness. At least, randomness is mandatory to decompose the input into shares. Moreover, many practical masked implementations require (large amounts of) fresh randomness during computation for masking gadgets and/or mask refreshing [38], [75], [85], [104]. Therefore, a pseudorandom generator (PRG) is a fundamental component of masked implementation, as essential as masked circuit¹. Common PRG choices in previous implementation include a linear feedback shift register (LFSR) [124], (reduced-round) block cipher (BC)-based extendable output function (XOF) [45], and stream cipher [26], [47].

However, their validity remains unclear due to a lack of detailed analysis. Discussions on the required security and properties of PRGs within the masking context are limited, whereas only the requirement that the generated mask should be uniform over the field/ring of the masking and cryptosystem is clearly understood. In masked implementation, the PRG output is not observable to the SCA attacker, while the PRG security has been evaluated generally in the plain model, assuming that the attacker can observe it. Alternatively, only the leakages of PRG are available in SCA. Nevertheless, the leakage resilience of PRGs have not been studied extensively in this context (Only Pietrzak [107] and Cassiers *et al.* [26] analyzed cases that attacker cannot observe PRG outputs but only observes the PRG leakages). Furthermore, current leakage assessment methodologies do not capture the aspects of mask-bit generation, which implies that there is currently no way to validate the practical security of masked implementations *experimentally* with regard to the mask-bit generator. Establishing a theoretical foundation of masking randomness generation is essential for practice of masked cryptographic implementation.

1. As in literature, we call an implementation of cryptographic function *circuit* for both hardware and software, because cryptographic algorithms are frequently implemented by logic operations.

1.2. Our contributions

In this study, we refer to pseudorandom bits for masking as *mask bits*. We present a systematic analysis of PRGs for mask-bit generation and an LR mask-bit generator.

Adversarial modeling. We establish three models—black-box, gray-box, and white-box models—which correspond to practical scenarios: leakage assessment, SCA success rate (SR) and key-lifetime evaluation [125], and practical deployment, respectively. Our three modelings constitute the first analytical framework to evaluate the mask-bit generation. Importantly, our three models are designed such that stronger adversaries strictly include weaker ones, which enables a stage-by-stage analysis of PRG requirement for secure mask-bit generation.

Three-stage analysis. We analyze three models in a stage-by-stage manner to identify the PRG properties to securely generate mask bits. In particular, we support a long-held belief/folklore with a proof and experiment: cryptographic security of PRG (*i.e.*, unpredictability) is sufficient but unnecessary for secure mask-bit generation². Instead, PRG output needs *high-dimensional uniform equidistribution*, which is easily generated by non-cryptographic PRGs (*e.g.*, LFSR). Additionally, in white-box model (*i.e.*, practical deployment), physical leakage resilience is essential.

New LR mask-bit generator. Based on our analyses, we propose an efficient LR mask-bit generation method that offers comparative performance with existing ones. It is on the basis of a concept of *leveled implementation* [17].

Brief summary. Table 1 summarizes the major PRGs and our proposals for mask-bit generation for practical deployment (See Section 6.4 for detailed discussion). Existing PRGs are impractical in terms of implementation cost and/or have difficulty in achieving a provable security with regard to leakage resilience. However, some PRGs may have a sufficiently practical security and performance. In Table 1, “No” and “Limited” in known SCA column mean that an SCA, such as simple power analysis (SPA), would be possible in principle but difficult in practice, as its success would be limited to only specific implementation and good signal-to-noise ratio (SNR). Meanwhile, “Practical” indicates that the SCA would be practical in major settings.

1.3. Related studies

In [58], Dziembowski and Pietrzak presented an LR stream cipher (DP), which achieves indistinguishability of the output from a random string under bounded leakage. DP was followed by several LR stream ciphers and PRGs to improve its efficiency [53], [107], [132]. LR-PRG/stream cipher may be promising for mask-bit generation from the security viewpoint, whereas its implementation performance is too inefficient for practice.

In [75, Section 6], Ishai *et al.* described how to construct a d -probing-secure PRG, which comprising a linear composition of d PRGs. Further, in [74], Ishai *et al.* presented a

robust PRG and showed a trivial construction with a linear composition of PRG and an improved construction based on an expander graph. Trivial and improved robust PRGs construct d -probing secure circuit with a cost of $O(d^4)$ and $O(d^{3+\epsilon})$ of true random number generator (TRNG) calls, respectively. In [37], Coron *et al.* improved the cost of TRNG calls to $O(d^2)$ based on a *multiple PRG* strategy (CGZ). However, the PRG(s) that should be used in the multiple PRG strategy was not discussed, as the PRG(s) are considered black-box and leak-free. Again, the leakage and SCA feasibility on PRGs have rarely been evaluated in the masking context. As we analyze them and reveal how to realize the PRG(s), our results would incorporate CGZ [37].

In [26], Cassiers *et al.* discussed hardware masking and claimed a vulnerability of LFSR in generating mask bits. Because the XOR of some bits of LFSR output is fixed to 0 owing to the construction of feedback polynomial, mask bits should be used while avoiding such a mask cancellation. They also demonstrated state-recovery SCA on LFSR for mask-bit generation as a major case study, which was in our white-box model. Yet, neither its generality nor formal security analysis was discussed. Besides, they evaluated the performance of PRGs for hardware masking. As our main focus is the security aspect of mask-bit generation, readers interested in PRG performance are advised to see [26].

1.4. Paper organization

Section 2 introduces the fundamentals of masking. Section 3 describes the modelings of masked implementation and attacks on them. Sections 4 to 6 analyze and discuss SCA on masked implementation in the black-, gray- and white-box models, respectively, and Section 5 includes an experimental validation of our claim. Section 7 provides our recommendations of PRGs to generate mask-bit for practitioners based on our analyses and proposes a novel LR primitive. Finally, Section 8 concludes this study.

2. Preliminaries

2.1. Notations

Uppercase and lowercase letters (*e.g.*, X and x) denote a random variable/vector and its realization, respectively, unless otherwise defined. Let $\mathbb{E}$ and p_X denote the expectation and probability mass or density function of X , respectively. We denote sequence of n random variables $X_1, X_2, \dots, X_n$ by X^n . We may write a keyed function as $F(K, \cdot) = F_K(\cdot)$.

2.2. Masking

Masking is a major countermeasure against SCA [28], [92]. In a masked implementation, each secret variable is decomposed into *shares* over an additive group defining the arithmetic operations of cryptosystem. In a d -th order masking over binary fields, a secret variable z is represented using $d + 1$ shares $a_0, a_1, \dots, a_d$ such that

$$z = a_0 \oplus a_1 \oplus \dots \oplus a_d.$$

2. For example, in [26], this proposition was assumed to be true without proof. This has been an important open-problem.

TABLE 1: Major symmetric primitives for mask-bit generation and our proposals (bold) under white-box SCA

Primitive	Typical instance	Implementation cost	Leakage resilience	Known SCA
Plain LFSR	Galois LFSR, Fibonacci LFSR	Low	No	Practical [24], [26], [80], [91], [119]
Improved LFSR	Mersenne twister [100], Xorshift [94]	Low	No	Probably limited [†]
Stream cipher	Trivium [40], [41]	Low	No	Limited [88]
LR-PRG/stream cipher	DP [58]	Very high	Yes	No (SPA only)
(Reduced-round) block cipher	CTR_DRBG [8], counter-based XOF	Medium	No	Practical [46], [79]
	OFB-based XOF	Medium	No	No (SPA only)
One-way hash	Hash_DRBG [8], counter-based XOF	Medium	No	Probably practical (same as CTR_DRBG)
	HMAC_DRBG [8], OFB-based XOF	Medium	No	Limited [81] (SPA only)
Permutation	Sponge-based XOF [19], [50]	Medium	Somehow [‡]	Limited [81] (SPA only)
(LR-)PRF-based XOF	GGM [65], FPS [62], MSJ [102]	Very high	Yes	No (SPA only)
	Sum of permutation [30], truncated BC [15]		Same as each XOF instance with block cipher	
Robust PRG	IKL+ [74]	Very high	d -probing	$(d+1)$ -probing
Multiple PRG	CGZ [37]	Low	d -probing*	$(d+1)$ -probing
Leveled mask-bit generator	Ascon + LFSR/stream cipher	Low	Yes	No (SPA only)
Masked stream cipher/XOF	Masking without fresh randomness	High	First order	Second-order blind SCA/SPA

[†] SCA on LFSR would be applicable to them in principle [24], [80], [91], [119], but it might be difficult to practical parallelized implementations.

[‡] Leakage resilience of sponge/duplex-based encryption is discussed in, e.g., [17], [18], [52].

* Security for masked circuit, but not for distinct PRG (See Section 6.2).

This is called *Boolean masking*³. For simplicity, we hereafter consider Boolean masking, which does not compromise the generality of our statements.

Masked linear operation is trivially implemented by duplicating the circuit for each share because for any linear function L and $z = a_0 \oplus a_1 \oplus \dots \oplus a_d$, it holds that $L(z) = L(a_0) \oplus L(a_1) \oplus \dots \oplus L(a_d)$. In contrast, it is non-trivial to implement masked non-linear circuit in a secure manner. For example, based on an Ishai–Sahai–Wagner (ISW) masking scheme [75], a masked AND gate for shared input $a = (a_0, a_1, \dots, a_d)$ and $b = (b_0, b_1, \dots, b_d)$ computes its output $c = (c_0, c_1, \dots, c_d)$ as follows: (i) draw random bits $m_{i,j}$ for $0 \leq i < j \leq d$ as fresh randomness, (ii) compute intermediate values $m_{j,i}$ as

$$m_{j,i} = (m_{i,j} \oplus a_i b_j) \oplus a_j b_i,$$

and (iii) compute the output share as

$$c_i = a_i b_i \oplus \bigoplus_{i \neq j} m_{i,j},$$

for each i . The security of this circuit is guaranteed as any d combination of probes does not reveal original (*i.e.*, non-shared) secret values. The ISW masking is followed by many masking schemes, such as (software) TI [64], [104], DOM [70], UMA [69], and gadget-based masking [10], for improved efficiency, enhanced security, composability, and resolving security degradation by physical defaults causing a cross-share interaction (*e.g.*, glitch [93], coupling [43], and microarchitectural leakage [63], [64]).

2.3. Probing attack models and notions

Attack models and security notions have been developed to soundly define the d -th order attack and masking scheme,

3. Our claim would hold for other advanced masking schemes, such as multiplicative [47], [66], inner-product [5], [56], code-based [31], [129], [130], and polynomial-interpolation maskings [68]. Note that, for maskings over non-binary fields/ring (*i.e.*, arithmetic masking [57], [67], [97]), the uniformity is defined over their field/ring, which has been used for secure implementation of lattice-based cryptography [1], [3], [4], [22], [33], [34], [35], [36], [48], [59], [105].

verification of masked implementation, *etc.* In the seminal study of [75], Ishai *et al.* presented the d -probing model, representing an attacker using d probes on the circuit. A masked circuit is d -probing secure if an adversary with a black-box access to the circuit can simulate a joint distribution of any d tuple of intermediate values; or simply and intuitively, any combination of d (non-adaptive) probes on the circuit recovers no secret variable [113]. The probing model has been extended in multiple aspects for more efficient and secure masking [10], [11], [27], [61], [112].

These probing-related models/notions are useful for defining a sound d -th order masking and verifying its implementation. Meanwhile, these models/notions do not directly represent realistic non-invasive SCAs, such as DPA, correlation power analysis (CPA) [23], and deep-learning based SCA (DL-SCA) [90], [96]. Fortunately, a security reduction to realistic model, namely noisy-leakage model, is known [54]. Thus, masked circuit based on these models/notions are currently promising for d -th order security.

Although methodologies of masked circuit are developing, how to generate mask bits has been rarely discussed. Only a few studies [37], [74], [75] showed constructions of d -probing secure PRG as described in Section 1.3; however, we mention a pitfall of d -probing model to evaluate SCA security of mask-bit generators in Section 6.2.

2.4. Security proof of masking

The above models/notions define d -th order masking and SCAs. As a sound security scheme, the SR on d -th order masked implementation should exponentially decrease by increasing d polynomially because an attacker on d -th order masked implementation is supposed to perform a $(d+1)$ -th order SCA. The first proof is shown by Chari *et al.* [28] for a single-bit DPA, assuming that the measurement noise is Gaussian and sufficiently large. It was followed and extended by several studies [12], [55], [77], [99], [109] to relax and generalize the conditions. Some recent proofs were based on an upper-bound of SR by a mutual information (*i.e.*, leakage amplitude or SNR of side-

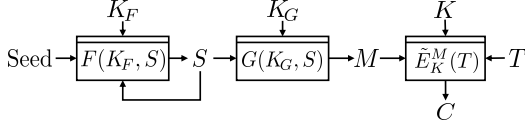


Figure 1: A model for masked implementation comprising masked circuit $\tilde{E}_K^M$ and PRG (F_{K_F}, G_{K_G}) .

channel measurement) [42], which gave proofs for general SCAs under practical conditions about noise.

These bounds hold assuming that the mask-bit generator is not attacked and the generated mask bits are ideally random. In practice, we should consider how to generate the random numbers with regard to SCA. However, there has been neither experimental strategy (through probing attack notions or leakage assessment) nor theoretical foundation to validate the practical security of masked implementations with a real-world (but non-ideal) PRG. This study solves the gap between theory and practice in mask-bit generation.

3. Modeling of SCA on masking

3.1. Modeling of masked implementation

Let n_f , n_g , n_s , and n_k be the bit lengths of internal state, generated mask bits per state, initial seed, and secret key, respectively. Figure 1 illustrates the abstract of masked implementation of encryption E_K with a PRG, where $\tilde{E}_K^M$ denotes a masked circuit of E_K that satisfies $\tilde{E}_K^M(T) = E_K(T)$ for any mask value M , key K , and plaintext T (C is ciphertext). The PRG consists of a state updating function F and output function G with secret keys K_F and K_G , respectively (keys can be *null*), and it has an n_f -bit internal state S computed from an n_s -bit initial seed. Let S_j be the j -th state, and M_j be the j -th output (*i.e.*, mask value). Given an n_s -bit seed, the PRG initializes the n_f -bit internal state as S_0 . The PRG updates the internal state as $S_{j+1} = F_{K_F}(S_j)$, while it generates an n_g -bit mask value as $M_j = G_{K_G}(S_j)$.

Example 3.1 (LFSR). The state updating function F is given by $S_{j+1} = \text{double}(S_j)$, where *double* is the doubling over $\text{GF}(2^{n_f})$ (n_f is the LFSR bit length), and the output function G is $\text{MSB}(S_j)$ (the most significant bit of S_j). Neither K_F nor K_G is given ($K_F = K_G = \text{null}$).

Example 3.2 (Counter-based XOF). Let H and E_{K_G} be a one-way hash function and BC with a key K_G , respectively. The initial state is given by an n_s -bit seed padded with an $(n_f - n_s)$ -bit zero ($n_s < n_f$). F is given by a counter as $S_{j+1} = S_j + 1$ and G is $H(S_j)$ or $E_{K_G}(S_j)$.

Remark 3.1 (Bit length of M and unrolling F and G). In practice, the bit length of M (*i.e.*, n_g) depends on the masked circuit implementation and the required number of mask bits per clock cycle. To generate a sufficient amount of mask bits for one block encryption, the PRG should repeatedly call F and G multiple times (or should be implemented in parallel). As in [26], a cryptographic hardware can be *unrolled* for higher throughput, implying an expansion of F and G . However, this has little impact on our black- and

gray-box analyses because it does not change the outputs essentially, whereas SCA resistance of the implementation would depend on unrolling (related to white-box model).

Remark 3.2 (Correlated randomness). For initial decomposition and refreshing in masked circuit, we need *correlated randomness* [2]; namely, d' random numbers $R_0, R_1, \dots, R_{d'}$ satisfying $R_0 \oplus R_1 \oplus \dots \oplus R_{d'} = 0$, where d' is the number of output shares. The initial decomposition of Z is given as $(A_0, A_1, \dots, A_d) = (R_0 \oplus Z, R_1, R_2, \dots, R_d)$ to satisfy the masking correctness: $Z = A_0 \oplus A_1 \oplus \dots \oplus A_d$. Refreshing is addition of shared zero to achieve uniformity while preserving the secret value. A typical way to generate such correlated randomness is to generate d random numbers $M_1, M_2, \dots, M_{d'}$ and then to give $R_i = M_i$ for $1 \leq i \leq d'$ and $R_0 = \bigoplus_{i=1}^{d'} M_i$ (If $d' = 1$, then $(R_0, R_1) = (M_1, M_1)$) or ring refreshing [10] realized as $R_i = R_i \oplus R_{i+1 \bmod d'}$.

3.2. Algorithmic leakage

If the PRG is inappropriate or flawed for mask-bit generator owing to its algorithmic nature, it is trivially broken. We identify two major algorithmic leakage, which would be exploited by a gray-box adversary.

Short frequency. If the attacker knows the frequency of PRG output, the attacker can trivially obtain traces with an identical mask and perform a first-order SCA using them. In practice, the mask-bit frequency should be sufficiently large so that such an attack gets infeasible, even for partial bits.

Brute-force. A first-order SCA trivially works using the mask bits as additional input(s) of the selection function, if the attacker can guess the mask bits with probability 1. Hence, the PRG to generate mask bits should have a sufficiently long bit length of secret key and/or seed, such that M is computationally hard-to-guess. Further, the state updating function should sufficiently diffuse the state to avoid a brute-force of partial state, which is discussed in Section 5.

3.3. Adversarial models

Let $\mathbf{X} \in \mathcal{X} \subset \mathbb{R}^l$ be side-channel trace, where l denotes the number of sample points. The SCA attacker always knows plaintext T and/or ciphertext C as in common SCAs, observes n traces $\mathbf{X}^n$ from the target device, and uses $(T, \mathbf{X})^n$ (or $(C, \mathbf{X})^n$) for key-recovery.

In SCA on masked implementation, the attacker observes the leakages from the d -th order masked circuit and/or the mask-bit generator. The attacker does not directly observe the mask bits (*i.e.*, PRG outputs) as in common masked implementation. As well, the attacker knows neither the seed nor secret key of PRG. Depending on the availability of leakage, we present three adversarial models of black-, gray-, and white-box (weaker to stronger in this order). Figure 2 displays the three models, where gray parts are components whose leakage is available by the attacker.

Black-box model (Figure 2a). The attacker has an access to only the leakage of the masked circuit $\tilde{E}_K^M$, but does not have access to leakages of mask bits M and mask-bit generators. As the relation to practical SCAs (*e.g.*, CPA)

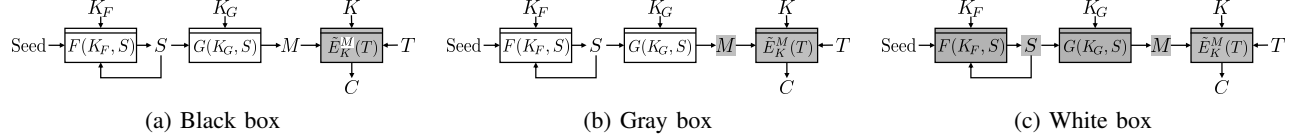


Figure 2: Adversarial models for masked cryptographic implementation, where gray parts (boundedly) leak to attacker.

or leakage assessment (e.g., TVLA [114]), the black-box attackers are defined using a limitation of *leakage and selection function* which the attacker can use; namely, the attacker is allowed to use only a selection and leakage function with *no input of M (or not more than d shares among $A_0, A_1, \dots, A_d$)*. For example, although the leakage of first-order masked implementation is frequently considered as $\text{HW}(A_0)$ and $\text{HW}(A_1)$ with noise, the black-box attacker is not allowed to take both A_0 and A_1 (or M) for the leakage and selection function. From the perspective of probing-related models, the black-box attacker can use one or up to d probe(s) on $\tilde{E}_K^M$, but cannot set the probe(s) on the mask-bit generator (i.e., F , G , and S), nor the generated mask M . Thus, this model corresponds to up-to d -th order SCA attacker on d -th order masked implementation and practical leakage assessment methodologies, which do not consider the leakage of mask-bit generator. Hence, if the implementation has no flaw in masked circuit, the black-box attack is never successful (i.e., the SR equals that of a random guess) and no leakage is detected; thus, this model is useful for discussing the required properties of M (i.e., PRG output for masking) apart from its leakage.

Gray-box model (Figure 2b). The attacker has an access to the leakages of M in addition to $\tilde{E}_K^M$, but still does not have access to the leakages of F and G . In the probing-related models, the gray-box attack corresponds to $d + 1$ probing on the d -th order masked circuit, while the attacker is not allowed to set probe(s) on the mask-bit generator. In practice, this model corresponds to security evaluation against higher-order SCAs, which do not utilize the PRG leakage as in the literature. Probing-related models/notions state that d -th order masked implementation is not secure against $(d+1)$ -probing attacks. However, practical adversary may take any d unboundedly; hence, it is important to consider the difficulty/feasibility of $(d + 1)$ -th order SCAs on d -th order masked implementation (typically as SR metric [121] and key lifetime [125]), apart from probing-related models. Additionally, if M leaks due to the algorithmic nature of PRG, the gray-box attacker may exploit it for mask guessing attack and biased mask attack [92], described in Section 5.4. Thus, the gray-box model is useful for discussing the masked circuit security quantitatively (i.e., SR and key-lifetime) and the algorithmic leakage.

White-box model (Figure 2c). A white-box attacker has an access to (or use any number of probes on) everywhere in the masked implementation, including F and G . The attacker can perform SCA on the mask-bit generator to retrieve its internal state and/or secret key. SCA without information of input nor output is sometimes called *blind SCA* [32], [89]. If they are retrieved, the attacker can guess

the mask bits with success probability 1; accordingly, the security order is degraded to zero as the attacker can perform first order SCA using guessed mask bits. This implies that, in the white-box model, the security of the entire masked implementation depends on the weaker of the PRG or the masked circuit. Therefore, it is important for masked implementation to evaluate physical leakage of PRG and assess its (in)vulnerability in practice. We comprehensively analyze the SCA security of major PRGs in the context of mask-bit generation in Section 6, while we provide a cautionary note in using probing-related models to evaluate the physical leakage of mask-bit generator.

3.4. Summary of our modeling and analyses: Take away messages

Let $\mathcal{B}$, $\mathcal{G}$, and $\mathcal{W}$ be sets of all black-, gray-, and white-box adversaries, respectively. Our models are designed in such a way that $\mathcal{B} \subset \mathcal{G} \subset \mathcal{W}$; namely, the set of gray-box (resp. white-box) adversaries truly includes the set of black-box (resp. gray-box) adversaries, as in Figure 2. This indicates that the proposition which holds in the black-box (resp. gray-box) model also holds in the gray-box (resp. white-box) model, because black-box (resp. gray-box) adversaries implies gray-box (resp. white-box) adversaries. Our modeling enables a stage-by-stage analysis of PRG properties for secure mask-bit generation.

Black-box model. It is defined such that the attacker does not use leakages of mask values (i.e., part of traces related to mask values), which is to discuss the required property of PRG outputs irrespective to its leakage. This corresponds to the leakage assessment, where evaluator wants to assess the leakage of masked circuit.

Gray-box model. Here, the leakage of mask bits (but not PRG) and potential attacks on masked circuit are discussed. The gray-box attacker performs a higher-order SCA on the masked circuit. In addition, the attacker may exploit the algorithmic leakage if inappropriate mask-bit generator is used, which enables to a mask-guessing first-order SCA (i.e., cancel the masking effect). This corresponds to the evaluation of resistance against higher-order SCA.

White-box model. Here, the PRG leakage is discussed. The attacker performs key/state-recovery SCA on PRG; its success enables first-order SCA with mask guessing. This corresponds to practical deployment, where any parts of implementation can be attacked. As the goal of key/state-recovery SCA on PRG is to mount a first-order mask-guessing SCA as well as gray-box SCA, the white-box SCA utilizes the physical leakage instead of algorithmic leakage if the algorithmic leakage is not exploitable.

TABLE 2: Summary of three models and analyses

Model	Discussion point(s)	Scenario	Required properties
Black	PRG output	Leakage assessment	Statistical uniformity
Gray	Higher-order SCA and AL of PRG	SR and key-lifetime evaluations	Statistical uniformity AL is unexploitable
White	Physical leakage of PRG	Practical deployment	Statistical uniformity AL is unexploitable Leakage resilience

Table 2 summarizes the discussion points and corresponding practical scenarios of three models, where AL means algorithmic leakage. Table 2 also shows the PRG properties required for secure mask-bit generation discussed in Sections 4 to 6. In all models/scenarios, the PRG output does not need cryptographic security (*i.e.*, unpredictability) for secure mask-bit generation, but a statistical uniformity (precisely, a high-dimensional uniform equidistribution) is sufficient, which can be achieved by, for example, LFSR. Cryptographic PRGs are also sufficient, as unpredictability implies statistical uniformity [131]. In gray- and white-box scenarios, the PRG additionally requires resistance to algorithmic or/and physical leakages to ensure the security.

4. Analysis of black-box adversary

4.1. Our key observation

If the masked circuit has no flaws and is soundly implemented (even with regard to physical defaults), the security order is degraded only if mask bits is inappropriate. As a trivial example, F and G are an identity function (*i.e.*, G outputs a constant value) or the output of G is highly biased far from uniform, in which the attacker would successfully recover the secret key solely by a first-order SCA, implying that the security order is degraded to zero no matter how large d is⁴. By contrast, it has been rarely discussed the properties that are required for mask-bit generation to achieve a secure masked implementation (*e.g.*, statistical randomness, computational unpredictability, or unreproducibility).

In [75], Ishai *et al.* mentioned that random bits are generated by a PRG, but did not mention the property that is required for the security of the ISW private circuit. In addition, in [104], Nikova *et al.* mentioned that the shares and mask should satisfy *uniformity* (*i.e.*, the occurrence probability of any value of shares is an identical constant independent of the secret value) and *statistical independence*, but did not mention whether the mask should be generated randomly. Furthermore, the security proof of masking [12], [13], [55], [77], [99] also assumed that shares are uniformly distributed and shares and leakages are independent. Some of these studies would only mention that uniformity and independence are mandatory for the security of masked circuit; however, (*psuedo or true*) *randomness*, *unpredictability*, or *unreproducibility* sometime has not been utilized

4. Note that the black-box attacker cannot leverage the knowledge of PRG output directly, as it is not allowed to take M as the input of selection function. Nevertheless, in this example, the first-order SCA is likely to be successful and leakage assessment would detect a first-order leakage.

to develop masking scheme or prove the security. In fact, unpredictability is sufficient but not necessary for generating secure mask bits. To prove it, we introduce Lemma 4.1 as our key observation, which states that the order of traces is meaningless for SCA and leakage assessment.

Lemma 4.1. *Given an attack dataset of side-channel traces and plaintexts/ciphertexts, the result of common leakage assessment methods and SCAs is invariant under reordering of pairs of traces and plaintexts/ciphertexts.*

Proof. This lemma follows from the fact that SCAs and leakage assessments utilize *sufficient statistics* such as empirical means, (co)variance, and related quantities (*e.g.*, likelihood), which considers the order of samples meaningless to represent the random variables/probability distribution. See Appendix A for the full proof. $\square$

4.2. Unnecessity of unpredictability

For a masked implementation, the trace order corresponds to the order of mask values (*i.e.*, how the mask bits are generated), as the j -th execution of masked circuit, from which attacker observes $(T_j, \mathbf{X}_j)$ ($1 \leq j \leq n$), uses the j -th mask value M_j . Lemma 4.1 implies that *the order of generated mask bits/value makes no sense for assessments and SCAs if empirical distribution of mask values is identical*. To be concrete, we introduce Theorem 4.1.

Theorem 4.1. *Let $\{(T_j, \mathbf{X}_j, M_j) \mid 1 \leq j \leq n\}$ be an attack dataset consisting of n plaintexts, n traces, and n mask values, where the j -th observation with $(T_j, \mathbf{X}_j)$ used mask value M_j . For any permutation $\pi_n : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$, SR of SCA using $\{(T_j, \mathbf{X}_j, M_{\pi_n(j)}) \mid 1 \leq j \leq n\}$ is same as that using $\{(T_j, \mathbf{X}_j, M_j) \mid 1 \leq j \leq n\}$.*

Proof. As we said, SCA and leakage assessment utilize sufficient statistic such as empirical means, unbiased empirical (co)variance, and likelihood. Hence, to prove Theorem 4.1, it suffices to show that the expectation of these sufficient statistics is invariant to π_n . Let f be a measurable function taking $\mathbf{X}$ (and T) as input, representing, for example, combining function in higher-order SCA. Its empirical mean with n traces conditioned by mask values $M^n = (M_0, M_1, \dots, M_{n-1})$ is expressed by

$$\begin{aligned} \mathbb{E}_{T^n, \mathbf{X}^n, M^n} [\hat{\mu}[f(T, \mathbf{X} \mid M)]] \\ = \frac{1}{n} \sum_j \mathbb{E}_{T, \mathbf{X}, M} f(T_j, \mathbf{X}_j \mid M_j). \end{aligned}$$

According to the definition of expectation, for any j , $\mathbb{E}_{T, \mathbf{X}, M} f(T_j, \mathbf{X}_j \mid M_j)$ is given by

$$\int \sum_{m_j} p_M(m_j) \sum_{t_j} p_{T, \mathbf{X}, M}(t_j, \mathbf{x}_j \mid m_j) f(t_j, \mathbf{x}_j \mid m_j) d\mathbf{x}_j.$$

In contrast, when the j -th mask value is given by $M_{\pi_n(j)}$, the empirical mean is expressed by $\mathbb{E}_{T, \mathbf{X}, M} f(T_j, \mathbf{X}_j \mid M_{\pi_n(j)})$, where m_j is solely replaced with $m_{\pi_n(j)}$. This replacement has no impact on the expectation, as we

take the summation about m_j or $m_{\pi_n(j)}$. Thus, it holds $\mathbb{E}_{T, \mathbf{X}, M} f(T_j, \mathbf{X}_j \mid M_{\pi_n(j)}) = \mathbb{E}_{T, \mathbf{X}, M} f(T_j, \mathbf{X}_j \mid M_j)$.

According to Lemma 4.1, the SCA/leakage assessment result is invariant to reordering. Because $\mathbb{E}_{T, \mathbf{X}, M} f(T_j, \mathbf{X}_j \mid M_{\pi_n(j)}) = \mathbb{E}_{T, \mathbf{X}, M} f(T_{\pi_n^{-1}(j)}, \mathbf{X}_{\pi_n^{-1}(j)} \mid M_j)$ holds, in expectation, the SCA/leakage assessment result is identical for $\{(T_j, \mathbf{X}_j, M_{\pi_n(j)}) \mid 1 \leq j \leq n\}$ and $\{(T_j, \mathbf{X}_j, M_j) \mid 1 \leq j \leq n\}$, which implies that the SRs are identical. $\square$

Using Theorem 4.1, we demonstrate that cryptographic security (*i.e.*, unpredictability) is unnecessary for secure mask-bit generation by constructing an extreme example. Assume that M_j takes a value from $\mathcal{M} = \{0, 1, \dots, n-1\}$ and mask-bit generator generates n different values during n cryptographic executions, where $n = 2^{n_g}$ (n_g is the bit length of M). Let π_n be a random permutation over $\{0, 1, \dots, n-1\}$. Consider two mask-bit generators with an upcounter and a cryptographic PRG using π_n , where the j -th bit mask value associated with $(T_j, \mathbf{X}_j)$ is $M_j = j$ and $\pi_n(j)$, respectively. The upcounter is not a cryptographically secure PRG. However, both methods have an identical empirical distribution uniform over $\{0, 1, \dots, n-1\}$. According to Theorem 4.1, the results of SCA are identical for both mask-bit generators based on upcounter and cryptographic PRG, as there exists a permutation $j \mapsto \pi_n(j)$ of Theorem 4.1. Thus, as far as $M^n = (M_1, M_2, \dots, M_n)$ is uniformly distributed over $\mathcal{M}^n$, the SCA SR on masked circuit is identical regardless of the order of mask values.

Because empirical sufficient statistics like mean play an essential role in SCA/assessment algorithms, it would be sufficient for output of mask-bit generator that the *empirical distribution* of mask bits/values is uniform. It is indeed natural, because, for example, TVLA evaluates the difference of empirical means two datasets. If the empirical distribution is identical, then the empirical mean is also identical. In summary, cryptographic security is unnecessary and non-cryptographic PRG is acceptable for mask-bit generation because the order of generated mask bits/value does not make a difference. In Section 5.5, we demonstrate an experimental second-order SCA on masked implementation with different mask-bit generators, which validates our claim in all the black-, gray-, and white-box settings; that is, the order of generated mask values has little impact on SCA result *on average* (*i.e.*, SR).

Remark 4.1 (Upcounter renders masked circuit insecure *in practice*). We showed the case of upcounter as a counter-intuitive example for a theoretical explanation. In practice, the empirical distribution of generated values is not uniform over $\{0, 1\}^{n_g}$ because 2^{n_g} (*i.e.*, frequency) is usually far greater than the number of traces n , implying that upper bits in M are hardly changed. This causes an algorithmic leakage exploitable by a gray-box attacker discussed in Section 5.3.

4.3. PRGs outputting secure mask bits

For generating statistical randomness yielding a uniform empirical distribution, we should consider methods that generate a high-dimensional uniform equidistribution.

Namely, for secure mask-bit generation, each bit/value of M should be unbiased (*i.e.*, uniformly equidistributed in one-dimension) as well as M should have neither auto-correlation nor bias in multiple bits; namely, it should be high-dimensional uniform equidistribution. Although it is impossible to generate *truly* uniform equidistribution in any finite dimension (even for cryptographic PRG), a fairly high-dimensional uniform equidistribution PRG has been commonly used, which would be sufficient for secure mask-bit generation. An example would be improved LFSRs like Mersenne twister [100], output of which is proven equidistributed uniformly over k -dimensional space with a 32-bit accuracy for all $1 \leq k \leq 623$. In addition, the outputs of an n_g -bit plain LFSR can be equidistributed uniformly in at most $(n_g - 1)$ -dimension with one-bit accuracy, which implies that LFSRs with a sufficiently large n_g would be sufficient for mask-bit generator. Cryptographic PRGs (*e.g.*, stream cipher and XOF) would be also appropriate, as unpredictability implies a high-dimensional uniform equidistribution. In contrast, the linear congruential generator in the standard library would be inappropriate, because it is not equidistributed uniformly in higher than one-dimension.

Relation to [26]. Cassiers *et al.* claimed that an LFSR-based mask-bit generator can lead to an insecurity owing to its linearity. Namely, according to the LFSR feedback polynomial, there exist mask bits that satisfy $M_{j_1} \oplus M_{j_2} \oplus \dots \oplus M_{j_c} = 0$ with probability 1, where M_{j_b} is the j_b -th bit of generated mask. If $M_{j_1}, M_{j_2}, \dots, M_{j_c}$ are XORed to obtain a correlated pseudorandomness, the resultant bit is fixed to zero, leading to an exploitable leakage. This implies that, in this case, the mask bits are not uniformly equidistributed even in one dimension owing to the construction of refreshing gadgets and defining polynomial. To ensure the security, a correlated pseudorandomness should be uniformly equidistributed with a care of linearity.

5. Analysis of gray-box adversary

5.1. Notations and graphical model

We introduce notations with a graphical model, which is an extension of a communication channel model representing SCA on masked implementation in [77]. Figure 3 shows the graphical model, where gray random variables are plaintext or leakage observable by the attacker. Note that this model does not represent an actual implementation; it represents relation among random variables (for example, Z does not appear in actual implementation). Here, random variables are defined as follows.

- $T^n = (T_1, T_2, \dots, T_n)$: (Partial) plaintext or ciphertext observable to attacker, where n is the number of traces.
- K : (Partial) secret key of masked circuit.
- $Z^n = (Z_1, Z_2, \dots, Z_n)$: Secret intermediate value calculated from K and T using a selection function. We may write $Z^{(k)}$ to specify the value of k . For example, $Z^{(k)} = \text{Sbox}(Z \oplus k)$ in some typical SCAs on AES.
- $M^n = (M_1, M_2, \dots, M_n)$: Generated mask bits.

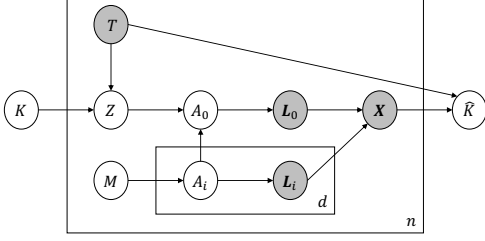


Figure 3: Graphical model representing gray-box SCA.

- A_i ($0 \leq i \leq d$): Shares of Z , where d is the order.
- L_i ($0 \leq i \leq d$): Leakage of A_i . We assume that leakages are independent, as in previous literature.
- $\mathbf{X}^n = (\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_n)$: Side-channel trace, each of which comprises leakages as $\mathbf{X} = (\mathbf{L}_i)_{i=0}^d$.
- $\hat{K}$: Estimation of (partial) secret key by SCA.

This graphical model represents that the secret variable is function of secret key K and plaintext/ciphertext T and side-channel trace $\mathbf{X}$ depends on Z . The plate of n represents that n traces are available for the attacker. In a masked implementation, Z is decomposed into shares $A_0, A_1, \dots, A_d$. Here, $Z \rightarrow A_0 \leftarrow A^d$ holds because the share decomposition is regarded as $(A_0, A_1, \dots, A_d) = (Z \oplus R_0, R_1, \dots, R_d)$, where $R_0, R_1, \dots, R_d$ are correlated randomness, and $A_i \rightarrow A_0$ represents the randomness correlation as $R_0 = R_1 \oplus R_2 \oplus \dots \oplus R_d$ holds. Note that the graphical model represents the relation among random variables, but does not represent any actual implementation. In other words, this model is valid regardless of actual implementation as long as the relation of random variables holds, which maintains the generality of the model. Leakages $L_0, L_1, \dots, L_d$ (where $\mathbf{X} = (L_0, L_1, \dots, L_d)$) correspond to $A_0, A_1, \dots, A_d$, respectively, and are assumed independent. Attacker estimates K as $\hat{K}$ from T^n and $\mathbf{X}^n$.

5.2. Optimal gray-box SCA

To discuss the capability and limitation of the gray-box adversary, we introduce a concept of *optimal distinguisher* [72], [78]. Let $(T^n, \mathbf{X}^n)$ be an attack dataset. Distinguisher $D(T^n, \mathbf{X}^n)$ is a function that returns an estimated secret key $\hat{K}$ using the attack dataset. The SR of SCA using n traces is defined as $\text{SR}_n(D) = \Pr(K = D(T^n, \mathbf{X}^n))$ [78], where $\Pr$ is the probability measure. Optimal distinguisher maximizes the SR.

Definition 5.1 (Optimal distinguisher [72], [78]). A distinguisher D_{opt} is optimal if its SR is the highest among distinguishers, namely, $\text{SR}_n(D_{\text{opt}}) = \sup_D \text{SR}_n(D)$.

We here introduce an optimal distinguisher that is used in DL-SCA⁵. Namely, a distinguisher using log-likelihood

5. Many constructions of optimal distinguisher are known [78]. For example, the template attack [29], which uses $p_{\mathbf{X}|T, K}$, is also optimal [72].

of secret key, defined as

$$D_{\text{opt}}(T^n, \mathbf{X}^n) = \arg \max_{k \in \mathcal{K}} \sum_j \log p_{Z|\mathbf{X}}(Z_j^{(k)} | \mathbf{X}_j), \quad (1)$$

is proven to be optimal [76], where $Z_j^{(k)}$ and $\mathbf{X}_j$ ($1 \leq j \leq n$) are the j -th secret intermediate value calculated from k and T_j and its side-channel trace, respectively.

The optimal distinguisher in Equation (1) is applicable to masked implementation; however, it does not represent the features and leakages of masking in a specific way. According to [77, Equation (2)], for d -th order masked implementation, $p_{Z|\mathbf{X}}$ can be represented using a convolution of conditional probability distributions of share given its leakage; namely, it holds that

$$p_{Z|\mathbf{X}}(z | \mathbf{X}) = \sum_{z=a_0 \oplus a_1 \oplus \dots \oplus a_d} \prod_{i=0}^d p_{A_i|L_i}(a_i | L_i), \quad (2)$$

where $p_{A_i|L_i}$ is the conditional probability distribution of j -th share A_j given its leakage L_i . Equation (2) represents *d convolution* of probability distributions⁶, which is essential to achieve an exponential security of masking [77]. As well, in the graphical model of Figure 3, we have Theorem 5.1 about an optimal distinguisher on masked implementation.

Theorem 5.1 (Optimal SCA on masked implementation [77], [78]). Consider the graphical model in Figure 3. A distinguisher defined as

$$\begin{aligned} D_{\text{opt}}(T^n, \mathbf{X}^n) &= \arg \max_{k \in \mathcal{K}} \sum_j \log p_{Z|\mathbf{X}}(Z_j^{(k)} | \mathbf{X}_j) \\ &= \arg \max_{k \in \mathcal{K}} \sum_j \log \sum_{Z_j^{(k)} = \bigoplus_i a_{i,j}} \prod_{i=0}^d p_{A_i|L_i}(a_{i,j} | L_{i,j}) \end{aligned}$$

is optimal, where $a_{i,j}$ and $L_{i,j}$ denote the i -th share of j -th execution and its leakage, respectively.

Proof. See Appendix B. $\square$

This optimal distinguisher is invariant under dataset reordering owing to commutativity of addition as same as Lemma 4.1. Therefore, a non-cryptographic PRG including LFSRs would be sufficient to evaluate a concrete security (e.g., SR and key-lifetime) of d -th order masked circuit against best possible gray-box SCA (i.e., $(d+1)$ -th order SCA), as far as its outputs are uniform.

5.3. Consideration of algorithmic leakage

For gray-box adversary, a best way to perform a key-recovery SCA is to use the optimal distinguisher. Here, if the mask-bit generator is inappropriate in some ways, the

6. For probability distributions on a finite discrete set, a convolution with a uniform distribution results in a uniform distribution. This implies that the optimal distinguisher in Theorem 5.1 is not successful in key recovery and has an SR of random guess if at least one leakage of share is not observable (namely, $p_{A_i|L_i} = p_{A_i}$ for some i), which corresponds to upto d -probing on d -th order masking (i.e., black-box attack).

attacker can estimate the values of M from its algorithmic nature, causing an algorithmic leakage. Note that any black-box attacker is not allowed to use it. Thus, the gray-box model is also useful for discussing the algorithmic leakage. Exploitation of algorithmic leakage enables the biased mask attack and mask guessing attack.

In fact, the upcounter mentioned in Section 4.2 is insecure in the gray-box model because of the algorithmic leakage, although the example provides the black-box security. First, the least significant bits (LSBs) of M_i has a very low frequency if using an upcounter. The gray-box attacker trivially knows collisions in LSBs; that is, for any j , it holds $\text{LSB}(M_{2j}) = \text{LSB}(M_{2(j+1)})$ with probability 1. This implies that a subset of M_j 's with only even or odd indices (*i.e.*, $\{M_{2j} \mid 1 \leq j \leq \lceil n/2 \rceil\}$ or $\{M_{2j-1} \mid 1 \leq j \leq \lceil n/2 \rceil\}$) is not uniform, and the d -th order masked implementation is not d -th order secure against SCAs using the corresponding subset. Such an SCA using a subset can also exploit MSBs, as the MSB is not changed during a consecutive $n/2$ M_j 's. Second, both updating and output functions (*i.e.*, F and G) have little diffusion effect. Suppose that the eight-bit LSBs of M is used for initial decomposition of first Sbox of first-order masked AES. Even if the initial seed is given at random, an eight-bit guess of the LSBs yields a hypothetical estimation of eight-bit LSBs of M_j for all j . Hence, an eight-bit guess of the seed would enable a successful first-order SCA on masked AES, regardless of the bit length of seed.

Quantitatively, the algorithmic leakage is unexploitable if the computational cost of mask-guessing grows exponentially by increase of number of traces (up to seed bit length). The computational cost of mask guessing of upcounter-based mask-bits generator is constant but does not increase by the number of traces. In contrast, standard PRGs including LFSR (with appropriate parameter) offer the exponential growth of the computational cost in mask guessing.

To formulate and guarantee such an exponential growth, a high-dimensional uniform equidistribution is useful again owing to its definition. High-dimensionally uniformly equidistributed M is guaranteed not to have partial bits with low frequency, namely, the first algorithmic leakage. Moreover, we must require at least n_{dim} -bit observation of outputs to predict the next outputs of an n_{dim} -dimensional uniform equidistribution, even if the PRG is not keyed. Because the PRG output is not observable, we need at least an n_{dim} -bit guess for a hypothetical estimation of (partial bits of) M_j for all j (when $n_{\text{total}} > n_{\text{dim}}$, where the required mask-bit length in total). The second algorithmic leakage gets computationally unavailable if n_{dim} is sufficiently large. Thus, both cryptographic and non-cryptographic PRGs including LFSR ($n_{\text{dim}} = n_g$) would be appropriate for mask-bit generator in the gray-box model as far as its output is a uniform equidistribution in fairly high dimensions.

5.4. Sophisticated gray-box attacks

Biased mask attack. Although other types of higher-order SCAs that utilize leakage of mask bits in a different

manner have been known, they cannot achieve higher performance than the optimal distinguisher. For example, a biased mask attack [92] selects traces with highly-biased or fixed mask bits, which is estimated by L_j or algorithmic leakage, and discards other traces. Estimation of highly-based mask bits corresponds to usage of $p_{A_i|L_i}$ or algorithmic leakage because this attack estimates the value of A_i . Meanwhile, discarding traces does not exploit their information, which results in an SR degradation. Thus, optimal distinguishers (*e.g.*, Theorem 5.1) achieve higher performance than the biased mask attack even for highly biased mask bits.

Mask guessing attack. If the attacker knows M in Figure 3 due to algorithmic leakage or white-box SCA on PRG, the attacker can guess the shares $a_1, a_2, \dots, a_d$ with probability 1 as $p_{A_i|M}$ and estimate k . The probability distribution in optimal distinguisher is expressed as

$$p_{Z|X,M}(z^{(k)} | X, m) = p_{A_0|L_0}(z^{(k)} \oplus a_1 \oplus \dots \oplus a_d | L_0),$$

which implies that the log-likelihood computation and key estimation can be performed without convolution. Thus, the attacker can perform a first-order SCA using L_0 and its SR gets identical to that on non-masked implementation. Consequently, optimal distinguishers achieve the best gray-box SCA with mask-bit exposure.

Fresh randomness leakage. The attacker may utilize a leakage of correlated pseudorandomness for mask refreshing. However, it would not improve the SR. Let us consider a refreshing of $A'_0, \dots, A'_d$ to $A_0, \dots, A_d$ using a correlated pseudorandomness $R_0, \dots, R_d$ as $A_i = A'_i \oplus R_i$. Note that both $A'_0, \dots, A'_d$ and $A_0, \dots, A_d$ represent identical unshared value Z . Let $L'_0, \dots, L'_d$ be their respective leakages. One way to use it would be using the probability distribution before refreshing by convoluting $p_{A_i|L_i}$ and $p_{R_i|L'_i}$ as

$$p_{A'_i|X}(a'_i | X) = \sum_{a'_i = a_i \oplus r_i} p_{A_i|L_i}(a_i | L_i) p_{R_i|L'_i}(r_i | L'_i),$$

to calculate the log-likelihood. However, it uses a convolution similar to Equation (2), which would degrade the SR, whereas mask refreshing (*i.e.*, adding shared zero) has no impact on the convolution in optimal distinguisher. Another way would be soft-analytical SCA (SASCA), which exploits all leakages in the masked circuit to recover secret key by belief propagation; however, it would not improve the SCA performance because (R_i, L'_i) is independent of Z and K .

5.5. Experimental validation

We demonstrate an experimental gray-box SCA, which validates the claim of Theorem 4.1. We simulate an optimal second-order SCA on first-order masked implementation with a mask-bit generator of cryptographic PRG or upcounter, where the attacker utilizes the leakage of shares as probability distribution. We also perform a mask-guessing first-order SCA when the mask-bit generator is upcounter. We targeted software AES implementation and focused on the first-round masked Sbox. We generated side-channel trace such that $L_i = \text{HW}(S_i) + N$, where $\text{HW}(S_i)$ is the

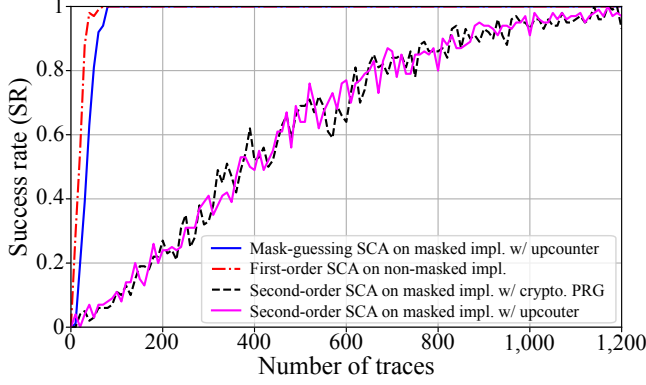


Figure 4: Experimental evaluation result of SCAs.

HW of S_i and N is a zero-mean Gaussian noise with variance $\sigma^2 = 3$. The shares S_0 and S_1 are given as $S_0 = Z \oplus M$ and $S_1 = M$. When evaluating upcounter, M is generated by an upcounter with a secret seed. Such leakage have been confirmed for many practical devices [92]. Note that we here focused on one Sbox instead of 128-bit SubBytes. Therefore, the issue about *low-frequency bits* mentioned in Remark 4.1 is irrelevant to this experiment when the number of traces is more than 256 (i.e., if $2^{n_g} < n$).

In this experiment, we evaluated the SR as a major evaluation metric [121] and is proven to decrease exponentially by an increase of masking order [55], [77]. We empirically evaluated the SR by repeating SCA trials 100 times and counting the number of successful key recovery. Figure 4 displays the SCA result, where the horizontal and vertical axes are the number of traces and SR, respectively⁷. As claimed, we found little difference of SR between mask-bit generators of cryptographic PRG and upcounter (minor difference would be caused by an empirical error). In contrast, the SR of mask-guessing first-order SCA offers an SR as high as first-order SCA on non-masked implementation, which means that the security order was degraded to zero due to the algorithmic leakage. Thus, we confirmed the validity of Theorem 4.1 and the claim on algorithmic leakage.

To validate our claim in the black-box setting, we also performed a computation of empirical mean of traces and performed the first-order TVLA. We found little difference in the results between mask-bit generators of cryptographic PRG and upcounter, and found no first-order leakage in both. Moreover, our claim is valid even in the while-box setting, because the white-box attacker exploits physical leakage of mask-bit generator (in addition to the algorithmic leakage) to perform the mask-guessing first-order SCA.

7. Due to the heavy computational cost, we evaluated mask-guessing SCA on masked impl. w/ upcounter using up to 200 traces. To perform mask-guessing with n traces, we need a computational cost of $O(2^{v(d+1)}n)$, where v is the Sbox bit-length ($v = 8$ for AES).

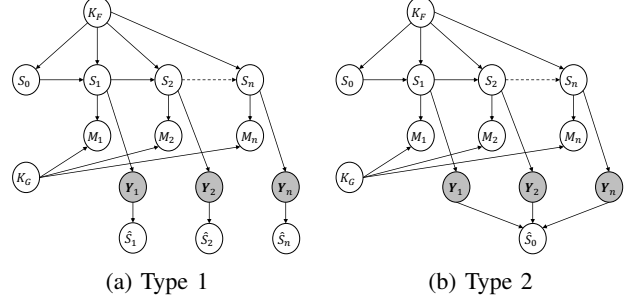


Figure 5: Graphical models representing white-box SCA.

6. Analysis of white-box adversary

6.1. Notations and graphical model

If the secret seed and key are exposed to the attacker, mask-guessing attack is enabled, meaning that the masked circuit is as in secure as non-masked one as evaluated in Section 5.5. We discuss (in)security of major PRGs against state/key-recovery SCA. See Table 1 for a summary. Figure 5 shows two graphical models representing an SCA on PRG, where estimated keys $\hat{K}_F$ and $\hat{K}_G$ from $\mathbf{Y}^n$ are omitted. The type depends on the property of F , as described in Section 6.3. Random variables are defined as follows:

- S_0 : Initial state of PRG given from seed.
- $S^n = (S_1, \dots, S_n)$: Internal states, where n is the number of traces. $\hat{S}_j$ is its estimation.
- K_F and K_G : Secret keys for updating function F and output function G , respectively (which can be null).
- $\mathbf{Y}^n = (Y_1, Y_2, \dots, Y_n)$: Mask-bit generator's leak.

This graphical model means that each state is generated from the previous state (and K_F), outputs are generated from the state (and K_G), and the attacker obtains the physical leakage of each state to estimate state or seed.

6.2. Adaptive attack on masking

In an SCA on masked cryptographic implementation, the attacker is supposed to first acquire n traces, including leakages of both masked circuit and PRG. Because the PRG and masked circuits operate distinctly, the attacker may first perform an SCA to recover the secret key and/or seed of the PRG; thereafter, the attacker can perform a first-order SCA with mask guessing if the attacker has successfully recovered them. In other words, a d -probing attacker can first use all probes on a PRG, then adaptively move the probes to use them on the masked circuit. This is trivially extended to the multiple PRG strategy [37], meaning that the attacker can adaptively use the probes on each PRG distinctly, as the PRGs operate distinctly. Although such an adaptive attack has been considered impractical for SCAs on masked circuit [62], [120], an adaptive SCA on PRG(s) would be practical if focusing on mask-bit generator.

The feasibility of adaptive SCA would indicate that, we should evaluate the SCA security of mask-bit generator

against practical SCAs, apart from probing-related models and masked circuit. In fact, we construct an example circuit that is d -probing secure but is vulnerable to an adaptive first-order SCA in Remark 6.1. Additionally, if we use probing-related model to evaluate the SCA security of PRGs, each PRG should be secure against d -probing. However, it may achieve higher security than required because only blind SCA is available owing to unobservability of M , which incurs considerable performance overhead.

6.3. Leakage models

The attacker uses a leakage from PRG to estimate state S , modeled as a conditional probability distribution $p_{S|Y}$, where Y is the leakage of M from PRG. The attacker obtains λ -bit information from a trace, where λ is given as mutual information $I(S; Y)$. This indicates that at most $n\lambda$ bits leaks as $I(S^n; Y^n) \leq nI(S; Y)$ if the attacker observes n traces. The mutual information represents *leakage amplitude* as in many previous works on LR cryptography [17], [49], [51], [52] and theoretical analyses of SR of SCA [12], [42], [55], [77], [98]. Depending on updating function F , we classify PRGs into two types.

Type 1 (F is a cryptographic primitive). In this type, F consists in a cryptographic primitive, such as BC and pseudorandom permutation (PRP). As the attacker is supposed to know neither secret key and seed, each state $S_1, S_2, \dots, S_n$ and their leakage $Y_1, Y_2, \dots, Y_n$ seem mutually independent owing to the indistinguishability. In this case, the attacker cannot perform DPAs using Y^n , but can only do an SPA that estimates S_j only from Y_j (as $Y_{j'}$ is independent of S_j for $j \neq j'$), as in Figure 5a. If the attacker observes n traces, at most $n\lambda$ bits leaks as $I(S^n; Y^n) = nI(S; Y)$, but S^n is considered to have $(n \times n_g)$ -bit information, which implies the difficulty in state-recovery SCA on this type. Although most symmetric primitive are not proven secure in the presence of leakage (except for LR ones [58], [106]), some type-1 primitives would be secure in practical settings.

Type 2 (F is not a cryptographic primitive). A major example of this type is a counter-based XOF with $F(S_j) = S_j + 1$. For this type of PRGs, $S_1, S_2, \dots, S_n$ and $Y_1, Y_2, \dots, Y_n$ are not independent even for the attacker. The attacker may perform key/state-recovery DPAs using n traces. Due to the dependence of S_j 's, the attacker may succeed in key recovery if he/she obtains n_f -bit information from leakages as $nI(S; Y) \leq I(S^n; Y^n)$. Thus, the attacker on type-2 may be able to estimate a state S_j from all leakages of Y^n . In fact, practical SCAs on some PRGs of this type are known, as discussed in Section 6.4.

6.4. SCA (in)security of major primitives

LFSRs. LFSR is type 2 with $F(S) = \text{double}(S)$ and $G(S) = \text{MSB}(S)$. The LFSR seed can be recovered if the attacker observes $2n_f$ -bit output of an n_f -bit LFSR. An SCA attacker obtains $I(S; Y)$ bits per trace and at most $n\lambda$ -bit information about the state from n traces. Thus, if the attacker observes n traces such that $n\lambda \geq 2n_f$, the attacker

may recover the seed or internal state. It would be difficult for LFSR to achieve a provable leakage resilience. Actually, state-recovery SCAs on plain LFSR (Galois and Fibonacci LFSRs) were reported in [24], [26], [80], [91], [119].

In contrast, improved LFSRs (e.g., Mersenne twister and Xorshift) are designed for a high parallelism, which consists of large internal state to realize a long frequency and high dimensionality. State recovery of such LFSRs would be significantly more challenging than plain LFSR, because it requires to observe far more output bits. As the improved LFSRs are implemented with a high parallelism, the leakage would contain less information than non-parallelized implementation due to algorithmic noise; the measurement with a high SNR would be difficult. Thus, although state recovery SCA would be possible *in principle*, improved LFSRs would be more secure than plain LFSRs *in practice*.

Remark 6.1 (Boolean masked LFSR is broken by first-order adaptive SCA). Because LFSR consists of solely linear operation, d -th order Boolean-masked LFSR is realized by d LFSRs. Here, trace X contains the leakage of each LFSR. In an n -trace SCA, the attacker first tries to recover the first LFSR state using the n traces. Then, attacker can also recover the states of second, third, $\dots$, and d -th LFSRs distinctly, as in [26, Section 4.3]. Thus, d repetition of adaptive first-order SCAs can recover d -probing secure LFSR state.

Stream cipher. It has been highly practical for generating pseudorandomness in terms of performance and implementation cost. For example, Trivium is one of major stream cipher [40] included in the ISO/IEC 29192-3 standard [73] and the portfolio of eSTREAM project [41], and it has been frequently utilized in masked implementations [26]. Stream cipher is type 2 as its round function is not PRP, and its leakage resilience is not proven generally. However, stream ciphers are designed in such a way that internal state cannot be recovered from output key stream; the state recovery would be more challenging than that of LFSR.

Kumar *et al.* reported a state recovery SCA on Trivium in [88]. However, its feasibility is limited to round-based implementation and very good SNR. Stream ciphers have been frequently implemented in an *unrolled* manner on both hardware and software for low energy [7], [25]⁸, and Kobayashi *et al.* claimed that the unrolling would improve the SCA security [86]. Although stream ciphers have difficulty in achieving a provable leakage resilience, their unrolled implementation would offer both a sufficient performance and practical security.

Meanwhile, some LR-PRGs/stream ciphers have been developed, which are proven to be secure in the indistinguishability notion under some leakage [58], [106]. However, their constructions much more complicated than non-LR stream ciphers and incurs considerably high implementation cost. Their security proofs assume that attacker observes its output directly. They would achieve a stronger security goal for mask-bit generation than required.

8. In implementing Trivium on a b -bit microcontroller, b bits are usually processed parallelly in a fixslicing manner (up to 64), which corresponds to a b -round unrolling.

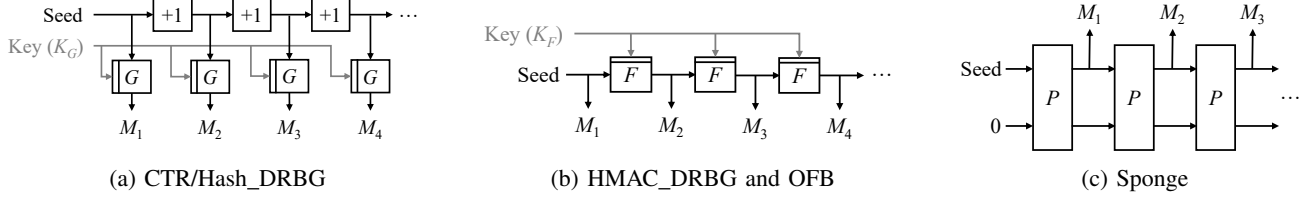


Figure 6: Extendable output functions (XOFs) based on cryptographic primitive.

XOFs. A typical PRG as an XOF is counter-based and type 2, which uses an upcounter for F and one-way hash H or BC E_{K_G} for output function G ; that is, $F(S_i) = S_i + 1$ and $G(S_i) = H(S_i)$ or $E_{K_G}(S_i)$, respectively. Figure 6a illustrates its overview, where $G = H$ or E_{K_G} . This construction is the basis of Hash_DRBG and CTR_DRBG, which are PRGs standardized in NIST SP 800-90A [8]. Counter-based XOFs do not have leakage resilience as it is type 2. In [46], de Meyer reported a practical blind SCA on CTR_DRBG based on the strategy on the counter-mode AES by Jaffe in [79], which achieved key and state recoveries within 256 traces and was improved by DL-SCA in [60]. The strategy would be trivially applied to counter-based XOFs with one-way hash including SHA-2 and SHA-3. Thus, counter-based XOF would be inappropriate for mask-bit generation.

A major way to construct a type-1 XOF from H or E_K is the output-feedback mode (OFB), as illustrated in Figure 6b, where $F = H$ or $F = E_{K_F}$. A NIST-standardized DRBG, HMAC_DRBG, is an instance that uses HMAC as $F(K_F, S_i) = \text{HMAC}_{K_F}(S_i)$. There is no known SCA except for SPA on it if the output is not observable for the attacker, as $S_1, S_2, \dots, S_n$ are mutually independent, which would imply that it is difficult to exploit their leakage jointly. In [81], Kannwischer *et al.* presented an input-recovery SASCA [128] on Keccak, which corresponds to state recovery of hash-based OFB; however, its success was limited to small microcontroller, bit-length of observable input (zero in our context), and/or good SNR. Thus, OFB-based XOF may be practically secure in generating mask bits. Note that a key-recovery blind SCA [32], [89] would be applicable to E_{K_F} in OFB; but mask guessing attack requires both key and state recovery.

Yet another major type-1 XOF construction would be sponge using a PRP, such as SHA-3 [19] and Ascon [50]. Figure 6c shows a sponge-based XOF, where P is an n_f -bit PRP. The updating and output functions are $F(S_i) = P(S_i)$ and $G(S_i) = \tau_r(S_i)$, respectively, where r is the rate and τ_r is an r -bit truncation. Leakage resilience of sponge-based cryptography has been discussed in [17], [49], [51], [52]. For example, ISAP [49], asakey [52], and Ascon [50] are claimed to be secure against SCAs if PRP is SPA resistant⁹. The above SASCA [81] is also applicable to this, while its feasibility was limited. Thus, sponge-based XOF would

offer a high SCA security as well as OFB-based ones.

Remark 6.2 (Reduced-round BC). A reduced-round BC may be used for XOFs in masking, as we do not require cryptographic PRG, which significantly improves the performance. In particular, an XOF based on 4-round AES has been frequently used if a non-cryptographic PRG is sufficient [82], for example, in UOV signature [20].

Pseudorandom function (PRF). One may use a PRF R_K to construct an XOF in Section 6.4 instead of BC or HMAC. The classical GGM would be a typical PRF [65], which was followed by two LR-PRFs, FPS and MSJ [62], [102] (and the leakage resilience of GGM was discussed in [102]). However, these PRF evaluation requires many calls cryptographic PRG, resulting in a non-negligible latency. They would have difficulty in achieving the practical performance as a mask-bit generator. Another major PRF constructions are sum of BC or PRP [30], [71], [118] (e.g., $R_{K_1, K_2}(T) = E_{K_1}(T) \oplus E_{K_2}(T)$) and truncation of BC or PRP [15]. Their performance would be comparable to BC or PRP; but, the practical advantage of non-invertibility, which may imply forward-secrecy, is unclear in masking context. Thus, PRF would be less appropriate than BC/PRP.

7. Our recommendations for practitioners

7.1. Black- and gray-box scenarios

Recall that black- and gray-box models correspond to leakage assessment and SCA resistance evaluation of masked circuit by designer/implementer, respectively, apart from the SCA security of PRG. Any PRG can yield valid assessment/evaluation if its output is statistically uniform. Hence, for assessment/evaluation, its availability and implementation costs would be of utmost importance. LFSRs and stream cipher would be useful owing to their high availability and performance (throughput and energy consumption).

7.2. White-box scenario

We have to consider SCA resistance of PRG because attackers can target the mask-bit generator. We list recommendations with respect to practical security and performance.

Improved LFSRs. Some LFSRs like Mersenne twister and Xorshift would have practical security against state-recovery SCAs. Their practical implementation exploits high parallelism and has a large state which renders the state-recovery SCA practically difficult. Yet, they would not have

9. Their leakage resilience is not formally proven; but, there are some discussions and positive results [17], [18], [52]. If we believe that their theory is true, the output is cryptographically secure and the module needs only SPA resistance to offer leakage resilience as an XOF.

a provable security, implying the feasibility of state recovery if a significant number of traces is observed. This motivates our proposal based on the LR reseeding in Section 7.3

Stream ciphers. Stream cipher, especially its unrolled implementation, would be very practical for mask-bit generation, as recommended by Cassiers *et al.* [26]. A b -round unrolled hardware implementation outputs b pseudorandom bits per clock cycle, and large unrolling degree improves the SCA security [86]. Banik *et al.* showed that a 288-round unrolling achieves highest energy efficiency [7]. As well, a software implementation on b -bit microcontroller usually employs a b -round unrolling using b -bit registers.

OFB- and sponge-based XOFs. Some type 1 XOFs (e.g., Ascon) are claimed to achieve an SCA security if the underlying primitive is SPA resistant. Although they would achieve a higher SCA security than improved LFSR and stream cipher, the performance might be worse, as these XOFs require to output random value M_j .

Masked stream ciphers/XOFs. First-order masking (e.g., TI) of some primitives can be implemented with practical performance without fresh randomness (e.g., Ascon [108], Trivium [115], Keccak [39], and AES [122]). Using them, we can realize a first-order secure mask-bit generator, because initial decomposition is not needed in the masking context once it is activated using a seed.

7.3. Proposed LR mask-bit generation method for white-box scenario

From performance perspective, (improved) LFSR would be best (or stream cipher would be acceptable), whereas type-1 XOFs would achieve a guarantee of high SCA security (i.e., leakage resilience). We present an LR mask-bit generation using *reseeding*, which is based on the concept of leveled implementation that combines LR and non-LR primitives to exploit their distinct advantages. The proposed method comprises a reseeding function and seed expander. The reseeding function, which generates a temporal seed from initial key, should be an LR cryptographic primitive, such as Ascon (a NIST-standardized primitive, claimed to be LR) [50], [123]. The seed expander is a non-LR high-performance PRG to expand the pseudorandomness (i.e., temporal seed) given from LR primitive. The expander is reseeded (i.e., initialized with a new seed) at a sufficiently high frequency to limit the number of traces available for SCA, similarly to rekeying [101], [125]. Our idea is based on the fact that LR primitives may not have high throughput whereas non-LR and non-cryptographic PRG can have. Their combination implies a leveled implementation, which achieves both leakage resilience and high performance.

Figure 7 displays the overview of proposed method. As a seed expander, we use a high-throughput PRG (e.g., improved LFSR) with a fixed output length, $\alpha : \{0, 1\}^{n_f} \rightarrow \{0, 1\}^{un_g}$ for some $un_g > n_f$, where n_f and n_g are bit lengths of LFSR state and M , respectively, and u is a reseeding interval unit. As a reseeding function, we use an LR primitive $\beta : \mathcal{K} \rightarrow \mathcal{K} \times \{0, 1\}^{n_f}$, where $\mathcal{K}$ is a key space. Here, β is used to reset α 's state (i.e., reseed) as well

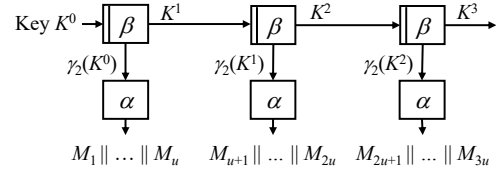


Figure 7: Proposed LR mask-bit generator, where α is seed expander (e.g., LFSR) and β is reseeding function by LR cryptographic primitive like Ascon.

as to perform rekeying of β . Let $\beta(K) = (\gamma_1(K), \gamma_2(K))$ and $K^j := \gamma_1^j(K)$ (γ_1^j denotes composition of j γ_1 's). Starting with initial uniformly random PRG key of β , denoted by K^0 , at each j -th step, we apply β to $\gamma_1(K^{j-1})$ and α to $\gamma_2(K^{j-1})$ and mask bits (values) $(M_{ju} \parallel \dots \parallel M_{(j+1)u}) = \alpha(\gamma_2(K^j))$. This produces the total output $\alpha(\gamma_2(K^0)) \parallel \alpha(\gamma_2(K^1)) \parallel \dots \parallel \alpha(\gamma_2(K^{j-1}))$ after the j -th reseeding step. The reseeding interval unit u (i.e., inverse of reseeding frequency) will be determined by considering security-efficiency tradeoff for the given improved LFSR and its implementation security. Namely, given state size of the LFSR n_f and leakage amplitude $\lambda = I(S; Y)$, we set u as reseeding interval/frequency such that $n_f \leq u\lambda$, where attacker cannot obtain n_f bits to recover the state for a seed.

This scheme has a similar structure to a forward-secure PRG by Bellare and Yee [16], whereas if we use LFSR to boost the throughput, the output is clearly not cryptographically secure. However, thanks to the LR security of β , we can regard the sequence $(M_{ju+1}, M_{ju+2}, \dots, M_{(j+1)u})$ for each j as an output sequence of α taking an independent and random seed in the presense of leakage. This means that the entire output sequence of M has sufficient properties as mask bits (i.e., high-dimensional uniform equidistribution and a low collision probability) up to the LR security of β .

8. Conclusion and future studies

This study provided a systematic analysis of PRGs for masked cryptographic implementation. We established three adversarial models to analyze the PRG properties required for secure mask-bit generation, and analyzed them to reveal the properties of PRGs required for secure mask-bit generation with an experimental validation. We then provided a comprehensive analysis of physical (in)security of major PRGs for practical deployment. Based on our analyses, we showed our recommendations and presented an LR mask-bit generator. Examples of the future studies are as follows:

Establishing methodology for experimental evaluation and validation. As mentioned in Section 1, it remains unknown how to evaluate and validate the mask-bit generation *experimentally*, because current SCA evaluation and leakage assessment do not fully capture the PRG properties as long as it yields a uniform empirical distribution. Our theoretical foundation would be useful to establish its methodology to evaluate and assess the practical security of whole masked implementation including PRG. Extending

the experiment in Section 5.5, which validated Theorem 4.1 and algorithmic leakage, would be a potential direction.

Detailed security analysis of proposed LR mask-bit generator. Our hybrid LR mask-bit generation method would be formally analyzed within the framework of provable security of leveled implementation [17] in combination with the discussion at Sections 4.2 and 6.4, which would make its security more concrete and would be useful to develop practical implementation with appropriate parameters.

References

- [1] A. Aikata, A. Basso, G. Cassiers, A. C. Mert, and S. Sinha Roy, “Kavach: Lightweight masking techniques for polynomial arithmetic in lattice-based cryptography,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2023, no. 3, pp. 366–390, 2023.
- [2] T. Araki, J. Furukawa, Y. Lindell, A. Nof, and K. Ohara, “High-throughput semi-honest secure three-party computation with an honest majority,” in *ACM CCS*, 2016, pp. 805–817.
- [3] M. Azouaoui, O. Bronchain, G. Cassiers, C. Hoffmann, Y. Kuzovkova, J. Renes, T. Schneider, M. Schönauer, F.-X. Standaert, and C. van Vredendaal, “Protecting Dilithium against leakage: Revisited sensitivity analysis and improved implementations,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, no. 4, pp. 58–79, 2023.
- [4] F. Bache, C. Paglialong, T. Oder, T. Schneider, and T. Güneysu, “High-speed masking for polynomial comparison in lattice-based KEMs,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2020, no. 3, pp. 483–507, 2020.
- [5] J. Balasch, S. Faust, B. Gierlichs, and I. Verbauwhede, “Theory and practice of a leakage resilient masking scheme,” in *ASIACRYPT*, 2012, pp. 758–775.
- [6] J. Balasch, B. Gierlichs, V. Grosso, O. Reparaz, and F.-X. Standaert, “On the cost of lazy engineering for masked software implementations,” in *CARDIS*, 2014, pp. 64–81.
- [7] S. Banik, V. Mikhalev, F. Armknecht, T. Isobe, W. Meier, A. Bogdanov, Y. Watanabe, and F. Regazzoni, “Towards low energy stream ciphers,” *IACR Trans. Symmetric Cryptol.*, no. 2, pp. 1–19, 2018.
- [8] E. Barker and J. Kelsey, “NIST special publication 800-90a: Recommendation for random number generation using deterministic random bit generators,” Jan. 2012, computer Security Division Information Technology Laboratory, National Institute of Standards and Technology. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-90a.pdf>
- [9] G. Barthe, S. Belaïd, G. Cassiers, P.-A. Fouque, B. Grégoire, and F.-X. Standaert, “maskVerif: Automated verification of higher-order masking in presence of physical defaults,” in *ESORICS*, 2019, pp. 300–318.
- [10] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, P.-Y. Strub, and R. Zucchini, “Strong non-interference and type-directed higher-order masking,” in *ACM CCS*, 2016.
- [11] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, and P.-Y. Strub, “Verified proofs of higher-order masking,” in *EUROCRYPT*, 2015, pp. 457–485.
- [12] J. Béguinot, W. Cheng, S. Guilley, Y. Liu, L. Masure, O. Rioul, and F.-X. Standaert, “Removing the field size loss from Duc et al.’s conjectured bound for masked encodings,” in *COSADE*, 2023, pp. 86–104.
- [13] J. Béguinot, W. Cheng, S. Guilley, and O. Rioul, “Formal security proofs via Doeblin coefficients: Optimal side-channel factorization from noisy leakage to random probing,” in *CRYPTO*, 2024.
- [14] S. Belaïd, D. Mercadier, M. Rivain, and A. R. Taleb, “IronMask: Versatile verification of masking security,” in *IEEE S&P*, 2022, pp. 142–160.
- [15] M. Bellare, T. Krovetz, and P. Rogaway, “Luby-Rackoff backwards: Increasing security by making block ciphers non-invertible,” in *EUROCRYPT*, 1998, pp. 266–280.
- [16] M. Bellare and B. S. Yee, “Forward-Security in Private-Key Cryptography,” in *CT-RSA*, ser. Lecture Notes in Computer Science, vol. 2612. Springer, 2003, pp. 1–18.
- [17] D. Bellizia, O. Bronchain, G. Cassiers, V. Grosso, C. Guo, C. Momin, O. Pereira, T. Peters, and F.-X. Standaert, “Mode-level vs. implementation-level physical security in symmetric cryptography: A practical guide through the leakage-resistance jungle,” in *CRYPTO*, 2020, pp. 369–400.
- [18] H. Berendsen and B. Mennink, “Tightening leakage resilience of the suffix keyed sponge,” *IACR Trans. Symmetric Cryptol.*, vol. 2024, no. 1, pp. 459–496, 2024.
- [19] G. Bertoni, J. Daemen, S. Hoffert, M. Peeters, G. Van Assche, and R. Van Keer, “Team Keccak,” Nov. 2023. [Online]. Available: <https://keccak.team>
- [20] W. Beullens et al., “UOV: Unbalanced oil and vinegar,” NIST Post-Quantum Cryptography, Aug 2024. [Online]. Available: <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/UOV-spec-web.pdf>
- [21] C. M. Bishop, *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006.
- [22] J. W. Bos, M. Gourjon, J. Renes, T. Schneider, and C. van Vredendaal, “Masking Kyber: First- and higher-order implementations,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2021, no. 4, pp. 173–214, 2021.
- [23] E. Brier, C. Clavier, and F. Olivier, “Correlation power analysis with a leakage model,” in *CHES*, 2004, pp. 16–29.
- [24] S. Burman, D. Mukhopadhyay, and K. Veezhinathan, “LFSR based stream ciphers are vulnerable to power attacks,” in *INDOCRYPT*, 2007, pp. 384–392.
- [25] A. Caforio, S. Banik, Y. Todo, W. Meier, T. Isobe, F. Liu, and B. Zhang, “Perfect trees: Designing energy-optimal symmetric encryption primitives,” *IACR Trans. Symmetric Cryptol.*, no. 4, pp. 36–73, 2021.
- [26] G. Cassiers, L. Masure, C. Momin, T. Moos, A. Moradi, and F.-X. Standaert, “Randomness generation for secure hardware masking – unrulled Trivium to the rescue,” *IACR Communications in Cryptology*, vol. 1, no. 2, 2024.
- [27] G. Cassiers and F.-X. Standaert, “Trivially and efficiently composing masked gadgets with probe isolating non-interference,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2542–2555, 2020.
- [28] S. Chari, C. S. Jutla, J. R. Rao, and P. Rohatgi, “Towards sound approaches to counteract power-analysis attacks,” in *CRYPTO*, 1999, pp. 398–412.
- [29] S. Chari, J. R. Rao, and P. Rohatgi, “Template attacks,” in *CHES*, 2002, pp. 13–28.
- [30] Y. L. Chen, E. Lambooi, and B. Mennink, “How to build pseudo-random functions from public random permutations,” in *CRYPTO*, 2019, pp. 266–293.
- [31] W. Cheng, S. Guilley, C. Carlet, J.-L. Danger, and S. Mesnager, “Information leakages in code-based masking: A unified quantification approach,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 3, pp. 465–495, 2021.
- [32] C. Clavier and L. Reynaud, “Improved blind side-channel analysis by exploitation of joint distributions of leakages,” in *CHES*, 2017, pp. 24–44.
- [33] J.-S. Coron, F. Gérard, S. Montoya, and R. Zeitoun, “High-order polynomial comparison and masking lattice-based encryption,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2023, no. 1, pp. 153–192, 2022.

- [34] —, “High-order table-based conversion algorithms and masking lattice-based encryption,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, no. 2, pp. 1–40, 2022.
- [35] J.-S. Coron, F. Gérard, M. Trannoy, and R. Zeitoun, “High-order masking of NTRU,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2023, no. 2, pp. 180–211, 2023.
- [36] —, “Improved gadgets for the high-order masking of Dilithium,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2023, no. 4, pp. 110–145, 2023.
- [37] J.-S. Coron, A. Greuet, and R. Zeitoun, “Side-channel masking with pseudo-random generator,” in *EUROCRYPT*, 2020, pp. 342–375.
- [38] J.-S. Coron, E. Prouff, M. Rivain, and T. Roche, “Higher-order side channel security and mask refreshing,” in *FSE*, 2014, pp. 410–424.
- [39] J. Daemen, “Changing of the guards: A simple and efficient method for achieving uniformity in threshold sharing,” in *CHES*, 2017, pp. 137–153.
- [40] C. De Canniere, “Trivium specifications,” 2006, http://www.ecrypt.eu.org/stream/p3ciphers/trivium/trivium_p3.pdf.
- [41] C. De Canniere and B. Preneel, “Trivium,” in *New Stream Cipher Designs: The eSTREAM Finalists*. Springer, 2008, pp. 244–266.
- [42] E. de Chérisey, S. Guilley, O. Rioul, and P. Piantanida, “Best information is most successful: Mutual information and success rate in side-channel analysis,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2019, no. 2, pp. 49–79, 2019.
- [43] T. de Cnudde, B. Bilgin, B. Gierlichs, V. Nikov, S. Nikova, and V. Rijmen, “Does coupling affect the security of masked implementations?” in *COSADE*, 2017, pp. 1–18.
- [44] T. De Cnudde, M. Ender, and A. Moradi, “Hardware masking, revisited,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, no. 2, pp. 123–148, 2018.
- [45] T. De Cnudde, O. Reparaz, B. Bilgin, S. Nikova, V. Nikov, and V. Rijmen, “Masking AES with $d+1$ shares in hardware,” in *CHES*, 2016, pp. 194–212.
- [46] L. De Meyer, “Recovering the CTR_DRBG state in 256 traces,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2020, no. 1, pp. 37–65, 2019.
- [47] L. De Meyer, O. Reparaz, and B. Bilgin, “Multiplicative masking for AES in hardware,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2018, no. 3, pp. 431–468, 2018.
- [48] R. del Pino, T. Prest, M. Rossi, and M.-J. O. Saarinen, “High-order masking of lattice signatures in quasilinear time,” in *IEEE S&P*, 2023, pp. 1168–1185.
- [49] C. Dobraunig, M. Eichlseder, S. Mangard, F. Mendel, B. Mennik, R. Primas, and T. Unterlaggauer, “Isap v2.0,” *IACR Trans. Symmetric Cryptol.*, vol. 2020, no. S1, pp. 390–416, 2020.
- [50] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, “ASCON: Lightweight authentication & hashing,” 2021, <https://ascon.iaik.tugraz.at/index.html>.
- [51] C. Dobraunig and B. Mennink, “Leakage resilience of the duplex construction,” in *ASIACRYPT*, 2019, pp. 225–255.
- [52] C. Dobraunig, B. Mennink, and R. Primas, “Leakage and tamper resilient permutation-based cryptography,” in *ACM CCS*, 2022, pp. 859–873.
- [53] Y. Dodis and K. Pietrzak, “Leakage-resilient pseudorandom functions and side-channel attacks on Feistel networks,” in *CRYPTO*, 2010, pp. 21–40.
- [54] A. Duc, S. Dziembowski, and S. Faust, “Unifying leakage models: From probing attacks to noisy leakage,” in *EUROCRYPT*, 2014, pp. 423–440.
- [55] A. Duc, S. Faust, and F.-X. Standaert, “Making masking security proofs concrete: Or how to evaluate the security of any leakage device,” in *EUROCRYPT*, 2015, pp. 401–429.
- [56] S. Dziembowski and S. Faust, “Leakage-resilient circuits without computational assumptions,” in *TCC*, 2012, pp. 230–247.
- [57] S. Dziembowski, S. Faust, and M. Skórski, “Optimal amplification of noisy leakages,” in *Theory of Cryptography*, 2016, pp. 291–318.
- [58] S. Dziembowski and K. Pietrzak, “Leakage-resilient cryptography,” in *FOCS*, 2008, pp. 293–302.
- [59] J.-P. D’Anvers, D. Heinz, P. Pessl, M. Van Beirendonck, and I. Verbauwhede, “Higher-order masked ciphertext comparison for lattice-based cryptography,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, no. 2, pp. 115–139, 2022.
- [60] K. Ebina, R. Ueno, and N. Homma, “Side-channel analysis against SecOC-compliant AES-CMAC,” *IEEE Trans. Circ. Syst. II: Express Briefs*, vol. 70, no. 10, pp. 3772–3776, 2023.
- [61] S. Faust, V. Grosso, S. Merino Del Pozo, C. Paglialonga, and F.-X. Standaert, “Composable masking schemes in the presence of physical defaults & the robust probing model,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2018, no. 3, pp. 89–120, 2018.
- [62] S. Faust, K. Pietrzak, and J. Schipper, “Practical leakage-resilient symmetric cryptography,” in *CHES*, 2012, pp. 213–232.
- [63] S. Gao, B. Marshall, D. Page, and E. Oswald, “Share-slicing: Friend or foe?” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2020, no. 1, pp. 152–174, 2020.
- [64] J. Gaspoz and S. Dhooghe, “Threshold implementations in software: Micro-architectural leakages in algorithms,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2023, no. 2, pp. 155–179, 2023.
- [65] O. Goldreich, S. Goldwasser, and S. Micali, “How to construct random functions,” *J. ACM*, vol. 33, no. 4, pp. 792–807, 1986.
- [66] J. D. Golić and C. Tymen, “Multiplicative masking and power analysis of AES,” in *CHES*, 2003, pp. 198–212.
- [67] L. Goubin, “A sound method for switching between Boolean and arithmetic masking,” in *CHES*, 2001, pp. 3–15.
- [68] L. Goubin and A. Martinelli, “Protecting AES with Shamir’s secret sharing scheme,” in *CHES*, 2011, pp. 79–94.
- [69] H. Gross and S. Mangard, “Reconciling $d+1$ masking in hardware and software,” in *CHES*, 2017.
- [70] H. Gross, S. Mangard, and T. Korak, “Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order,” in *TIS, ACM CCS Workshops*, 2016, p. 3.
- [71] C. Hall, D. Wagner, J. Kelsey, and B. Schneier, “Building PRFs from PRPs,” in *CRYPTO*, 1998, pp. 370–389.
- [72] A. Heuser, O. Rioul, and S. Guilley, “Good is not good enough: Deriving optimal distinguishers from communication theory,” in *CHES*, 2014, pp. 55–74.
- [73] International Organization for Standardization, “ISO/IEC 29192-3:2012 Information technology—Security techniques—Lightweight cryptography— Part 3: Stream ciphers,” <https://www.iso.org/standard/56426.html>, Nov. 2023.
- [74] Y. Ishai, E. Kushilevitz, X. Li, R. Ostrovsky, M. Prabhakaran, A. Sahai, and D. Zuckerman, “Robust pseudorandom generators,” in *ICALP*, 2013, pp. 576–588.
- [75] Y. Ishai, A. Sahai, and D. Wagner, “Private circuits: Securing hardware against probing attacks,” in *CRYPTO*, 2003, pp. 463–481.
- [76] A. Ito, R. Ueno, and N. Homma, “Toward optimal deep-learning based side-channel attacks: Probability concentration inequality loss and its usage,” Cryptology ePrint Archive, Report 2021/1216, 2021, <https://ia.cr/2021/1216>.
- [77] —, “On the success rate of side-channel attacks on masked implementations: Information-theoretical bounds and their practical usage,” in *ACM CCS*, 2022, pp. 1521–1535.

- [78] —, “Perceived information revisited: New metrics to evaluate success rate of side-channel attacks,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 4, pp. 228–254, 2022.
- [79] J. Jaffe, “A first-order DPA attack against AES in counter mode with unknown initial counter,” in *CHES*, 2007, pp. 1–13.
- [80] A. Joux and P. Delaunay, “Galois LFSR, embedded devices and side channel weaknesses,” in *INDOCRYPT*, 2006, pp. 436–451.
- [81] M. J. Kannwischer, P. Pessl, and R. Primas, “Single-trace attacks on Keccak,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2020, no. 3, pp. 243–268, 2020.
- [82] L. Keliher and J. Sui, “Exact maximum expected differential and linear probability for two-round Advanced Encryption Standard,” *IET Inf. Secur.*, vol. 1, no. 2, pp. 53–57, 2007.
- [83] D. Knichel, A. Moradi, N. Müller, and P. Sasdrich, “Automated generation of masked hardware,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 1, pp. 589–629, 2022.
- [84] D. Knichel, P. Sasdrich, and A. Moradi, “SILVER—statistical independence and leakage verification,” in *ASIACRYPT*, 2020, pp. 787–816.
- [85] —, “Generic hardware private circuits: Towards automated generation of composable secure gadgets,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, pp. 323–344, 2021.
- [86] S. Kobayashi, R. Ueno, Y. Todo, and N. Homma, “Side-channel linearization attack on unrolled Trivium hardware,” *IACR Commun. Cryptol.*, vol. 1, no. 3, 2024.
- [87] P. Kocher, J. Jaffe, and B. Jun, “Differential power analysis,” in *Advances in Cryptology—CRYPTO’99*, ser. Lecture Notes in Computer Science, vol. 1666, 1999, pp. 388–397.
- [88] S. Kumar, V. A. Dasu, A. Bakshi, S. Sarkar, D. Jap, J. Breier, and S. Bhasin, “Side channel attack on stream ciphers: A three-step approach to state/key recovery,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, no. 2, pp. 166–191, 2022.
- [89] Y. Linge, C. Dumas, and S. Lambert-Lacroix, “Using the joint distributions of a cryptographic function in side channel analysis,” in *COSADE*, 2014, pp. 199–213.
- [90] H. Maghrebi, T. Portigliatti, and E. Prouff, “Breaking cryptographic implementations using deep learning techniques,” *SPACE*, pp. 3–26, 2016.
- [91] J. Maillard, A. H. Meraneh, M. Sarry, C. Clavier, H. L. Boudier, and G. Thomas, “Blind side channel analysis on the Elephant LFSR extended version,” in *E-Business and Telecommunications*, 2023, pp. 20–42.
- [92] S. Mangard, E. Oswald, and T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer New York, 2007.
- [93] S. Mangard, N. Pramstaller, and E. Oswald, “Successfully attacking masked AES hardware implementations,” in *CHES*, 2005, pp. 157–171.
- [94] G. Marsaglia, “Xorshift RNGs,” *J. Statistical Sof.*, vol. 8, no. 14, pp. 1–6, 2003.
- [95] B. Marshall, D. Page, and J. Webb, “MIRACLE: MiRo-Architectural Leakage Evaluation: A study of micro-architectural power leakage across many devices,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, no. 1, pp. 175–220, 2021.
- [96] L. Masure, C. Dumas, and E. Prouff, “A comprehensive study of deep learning for side-channel analysis,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 1, pp. 348–375, 2020.
- [97] L. Masure, P. Méaux, T. Moos, and F.-X. Standaert, “Effective and efficient masking with low noise using small-Mersenne-prime ciphers,” in *EUROCRYPT*, 2023, pp. 596–627.
- [98] L. Masure, O. Rioul, and F.-X. Standaert, “A nearly tight proof of Duc et al.’s conjectured security bound for masked implementations,” in *CARDIS*, 2022, pp. 86–104.
- [99] —, “A nearly tight proof of Duc et al.’s conjectured security bound for masked implementations,” in *Smart Card Research and Advanced Applications*, 2023, pp. 69–81.
- [100] M. Matsumoto and T. Nishimura, “Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM Trans. Model. Comput. Simul.*, vol. 8, no. 1, pp. 3–30, 1998.
- [101] M. Medwed, F. Standaert, J. Großschädl, and F. Regazzoni, “Fresh re-keying: Security against side-channel and fault attacks for low-cost devices,” in *Progress in Cryptology—AFRICACRYPT 2010*, ser. Lecture Notes in Computer Science, vol. 6055. Springer, 2010, pp. 279–296.
- [102] M. Medwed, F. Standaert, and A. Joux, “Towards super-exponential side-channel security with efficient leakage-resilient PRFs,” in *CHES*, 2012, pp. 193–212.
- [103] N. Müller, D. Knichel, P. Sasdrich, and A. Moradi, “Transitional leakage in theory and practice: Unveiling security flaws in masked circuits,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2, pp. 266–288, 2022.
- [104] S. Nikova, V. Rijmen, and M. Schläffer, “Secure hardware implementation of nonlinear functions in the presence of glitches,” *J. Cryptol.*, vol. 24, no. 2, pp. 292–321, 2011.
- [105] T. Oder, T. Schneider, T. Pöppelmann, and T. Güneysu, “Practical CCA2-secure and masked ring-LWE implementation,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2018, no. 1, pp. 142–174, 2018.
- [106] O. Pereira, F.-X. Standaert, and S. Vivek, “Leakage-resilient authentication and encryption from symmetric cryptographic primitives,” in *ACM CCS*, 2015, pp. 96–108.
- [107] K. Pietrzak, “A leakage-resilient mode of operation,” in *EUROCRYPT*, 2009, pp. 462–482.
- [108] S. H. Prasad, F. Mendel, M. Schläffer, and R. Nagpal, “Efficient low-latency masking of ascon without fresh randomness,” *Cryptology ePrint Archive*, Paper 2023/1914, 2023.
- [109] E. Prouff and M. Rivain, “Masking against side-channel attacks: A formal security proof,” in *EUROCRYPT*, 2013, pp. 142–159.
- [110] M. Renaud, F.-X. Standaert, N. Veyrat-Charvillon, D. Kamel, and D. Flandre, “A formal study of power variability issues and side-channel attacks for nanoscale devices,” in *EUROCRYPT*, 2011, pp. 109–128.
- [111] O. Reparaz, “Detecting flawed masking schemes with leakage detection tests,” in *FSE*, 2016, pp. 204–222.
- [112] O. Reparaz, B. Bilgin, S. Nikova, B. Gierlichs, and I. Verbauwhede, “Consolidating masking schemes,” in *Advances in Cryptology—CRYPTO 2015*, 2015, pp. 764–783.
- [113] M. Rivain and E. Prouff, “Provably secure higher-order masking of AES,” in *CHES*, 2010, pp. 413–427.
- [114] T. Schneider and A. Moradi, “Leakage assessment methodology: A clear roadmap for side-channel evaluations,” in *CHES*, 2015, pp. 495–513.
- [115] D. Shanmugam and S. Annadurai, “Secure implementation of stream cipher: Trivium,” in *SECITC*, 2015, pp. 253–266.
- [116] A. M. Shelton, Ł. Chmielewski, N. Samwel, M. Wagner, L. Batina, and Y. Yarom, “Rosita++: Automatic higher-order leakage elimination from cryptographic code,” in *ACM CCS*, 2021, pp. 685–699.
- [117] A. M. Shelton, N. Samwel, L. Batina, F. Regazzoni, M. Wagner, and Y. Yarom, “Rosita: Towards automatic elimination of power-analysis leakage in ciphers,” in *NDSS*, 2021.
- [118] F. Sibleyras and Y. Todo, “Keyed sum of permutations: A simpler RP-based PRF,” in *CT-RSA*, 2023, pp. 573–593.
- [119] S. M. Sim, D. Jap, and S. Bhasin, “DAPA: Differential analysis aided power attack on (non-)linear feedback shift registers,” *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2021, no. 1, p. 169–191, 2020.

- [120] F.-X. Standaert, O. Pereira, Y. Yu, J.-J. Quisquater, M. Yung, and E. Oswald, "Leakage resilient cryptography in practice," in *Towards Hardware-Intrinsic Security: Foundations and Practice*, A.-R. Sadeghi and D. Naccache, Eds., 2010, pp. 99–134.
- [121] F.-X. Standaert, T. G. Malkin, and M. Yung, "A unified framework for the analysis of side-channel key recovery attacks," in *Advances in Cryptology—EUROCRYPT 2009*, ser. Lecture Notes in Computer Science, vol. 5479, 2009, pp. 443–461.
- [122] T. Sugawara, "3-share threshold implementation of aes s-box without fresh randomness," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2019, no. 1, pp. 123–145, 2018.
- [123] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Nist special publication 800 NIST SP 800-232 ipd: Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions," 2024, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-232.ipd.pdf>.
- [124] R. Ueno, N. Homma, and T. Aoki, "Toward more efficient DPA-resistant AES hardware architecture based on threshold implementation," in *COSADE*, 2017, pp. 50–64.
- [125] R. Ueno, N. Homma, A. Inoue, and K. Minematsu, "Fallen sanctuary: A higher-order and leakage-resilient rekeying scheme," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 1, no. 2024, pp. 264–308, 2023.
- [126] R. Ueno, N. Homma, S. Morioka, and T. Aoki, "Automatic generation of formally-proven tamper-resistant Galois-field multipliers based on generalized masking scheme," in *DATE*. IEEE, 2017, pp. 978–983.
- [127] —, "A systematic design methodology of formally proven side-channel-resistant cryptographic hardware," *IEEE Design & Test*, vol. 38, no. 3, pp. 84–92, 2021.
- [128] N. Veyrat-Charvillon, B. Gérard, and F.-X. Standaert, "Soft analytical side-channel attacks," in *ASIACRYPT*, 2014, pp. 282–296.
- [129] W. Wang, P. Méaux, G. Cassiers, and F.-X. Standaert, "Efficient and private computations with code-based masking," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2020, no. 2, pp. 128–171, 2020.
- [130] Q. Wu, W. Cheng, S. Guilley, F. Zhang, and W. Fu, "On efficient and secure code-based masking: A pragmatic evaluation," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 3, pp. 192–222, 2022.
- [131] A. C. Yao, "Theory and applications of trapdoor functions," in *IEEE Symposium on Foundations of Computer Science (FOCS)*, 1982, pp. 80–91.
- [132] Y. Yu, F.-X. Standaert, O. Pereira, and M. Yung, "Practical leakage-resilient pseudorandom generators," in *ACM CCS*, 2010, pp. 141–151.
- [133] J. Zeitschner and A. Moradi, "PoMMES: Prevention of micro-architectural leakages in masked embedded software," *IACR TCHES*, vol. 2024, no. 3, pp. 342–376, 2024.
- [134] J. Zeitschner, N. Müller, and A. Moradi, "PROLEAD_SW: Probing-based software leakage detection for ARM binaries," *IACR TCHES*, vol. 2023, no. 3, pp. 391–421, 2023.

Appendix A.

Proof of Lemma 4.1

Proof. Before the proof main body, we introduce notations, TVLA, and common SCAs in a formal description.

Notations. Let $X_1, X_2, \dots, X_n$ be n samples of random variable X . Its empirical mean and unbiased empirical variance are denoted by

$$\hat{\mu}[X] = \frac{1}{n} \sum_j X_j, \quad \hat{\sigma}^2[X] = \frac{1}{n-1} \sum_j (X_j - \hat{\mu}[X])^2,$$

respectively. Let $Y_1, Y_2, \dots, Y_n$ be a jointly sampled random variables of Y as well. Unbiased empirical covariance between X and Y is given by

$$\hat{\sigma}[X, Y] = \frac{1}{n-1} \sum_j (X_j - \hat{\mu}[X])(Y_j - \hat{\mu}[Y]).$$

Assume that, for $(d-1)$ -st order masked software implementation, a trace $\mathbf{X}$ consists of d leakages $L_1, L_2, \dots, L_d \in \mathbb{R}$ corresponding to its share $A_1, A_2, \dots, A_d$, respectively. In contrast, for $(d-1)$ -st masked hardware implementation, assume that the leakage is univariate and a trace $\mathbf{X} = L$ is given as sum of $L_1, L_2, \dots, L_d$. In the d -th order TVLA/SCA, we use d -th order leakage denoted by $V^{(d)}$. For d -variate SCA (typically on software implementation), the d -th order leakage is given by a product of zero-averaged d sample points as $V^{(d)} = \prod_{i=1}^d (L_i - \hat{\mu}[L_i])$. For univariate TVLA/SCA (typically on hardware implementation), the d -th order leakage is given by the d -th power of a zero-averaged sample point as $V^{(d)} = (L - \hat{\mu}[L])^d$.

Test vector leakage assessment (TVLA). A d -th order TVLA assesses the existence of d -th order leakage using a non-parametric t -test [114]. The TVLA is designed to reject a null hypothesis that mean of d -th order leakage is not different when the input plaintexts are a fixed value or random. For the d -th order TVLA, the empirical t -value is computed using d -th order leakage in traces as

$$\hat{t} = \frac{\hat{\mu}[V_{\text{fix}}^{(d)}] - \hat{\mu}[V_{\text{rnd}}^{(d)}]}{\sqrt{\frac{\hat{\sigma}^2[V_{\text{fix}}^{(d)}]}{n_{\text{fix}}} + \frac{\hat{\sigma}^2[V_{\text{rnd}}^{(d)}]}{n_{\text{rnd}}}}},$$

where $V_{\text{fix}}^{(d)}$ and $V_{\text{rnd}}^{(d)}$ are random variables of d -th order leakage with fixed and random plaintexts, respectively, and n_{fix} and n_{rnd} are the respective numbers of traces. If the resultant t -value is higher than threshold, the null hypothesis is rejected and the alternative hypothesis is accepted, implying that there exists a d -th order leakage. Otherwise, the null hypothesis is (negatively) accepted. A double-sided test with a significance level of 10^{-5} have been commonly used, which corresponds to absolute t -value threshold of 4.5.

Correlation power analysis (CPA). Let $\phi : \{0, 1\}^{n_s} \rightarrow \mathbb{R}$ be a leakage function (e.g., Hamming weight), where n_s denotes bit length of Z . A d -th order CPA estimates the secret key as $\hat{k}$ using Pearson's correlation coefficient $\hat{\rho}$ as

$$\begin{aligned} \hat{k} &= \arg \max_k \left| \hat{\rho}[\phi(Z^{(k)}), V^{(d)}] \right| \\ &= \arg \max_k \left| \frac{\hat{\sigma}[\phi(Z^{(k)}), V^{(d)}]}{\hat{\sigma}[\phi(Z^{(k)})] \cdot \hat{\sigma}[V^{(d)}]} \right|. \end{aligned}$$

Profiled deep-learning based SCA (DL-SCA). Let $q_{\hat{\theta}}$ be a trained neural network as a profiled leakage model, which imitates $p_{Z|X}$. A DL-SCA using the neural network estimates the secret key as

$$\hat{k} = \arg \max_k \hat{\mu}[\log q_{\hat{\theta}}(Z^{(k)} | \mathbf{X})].$$

Main body of proof. Common SCAs and TVLA compute the results solely using empirical mean, unbiased empirical variance, and unbiased empirical covariance, as described. These statistics are obviously invariant to the order of joint samples in $(T^n, \mathbf{X}^n)$ due to commutativity of addition. Thus, the leakage assessment and SCA results are invariant to the trace order. $\square$

Appendix B.

Proof of Theorem 5.1

Proof. Let $\mathbb{E}$ be the expectation and $\Pr$ be the probability measure. Let $\ell(\hat{k}, k) = \mathbb{1}_{\{\hat{k} \neq k\}}$ be a loss function, where $\mathbb{1}$ is the indicator function. Optimal distinguisher is defined as a distinguisher that minimizes the loss function ℓ in guessing the correct key. Taking the expectation of $\ell(D(T^n, \mathbf{X}^n), K)$, we have

$$\begin{aligned}\mathbb{E}\ell(D(T^n, \mathbf{X}^n), K) &= \mathbb{E}\mathbb{1}_{\{D(T^n, \mathbf{X}^n) \neq K\}} \\ &= \mathbb{E}\mathbb{E}[\mathbb{1}_{\{D(T^n, \mathbf{X}^n) \neq K\}} \mid T^n, \mathbf{X}^n] \\ &= \mathbb{E}[1 - \Pr(K = D(T^n, \mathbf{X}^n) \mid T^n, \mathbf{X}^n)].\end{aligned}$$

Therefore,

$$\begin{aligned}D(T^n, \mathbf{X}^n) &= \arg \max_k \Pr(K = k \mid T^n, \mathbf{X}^n) \\ &= \arg \max_k p_{K \mid \mathbf{X}^n, T^n}(k \mid T^n, \mathbf{X}^n)\end{aligned}$$

is an optimal distinguisher.

Then, we focus on a joint probability distribution $p_{K, \mathbf{X}^n, T^n}$ since $\arg \max_k p_{K \mid \mathbf{X}^n, T^n}(k \mid T^n, \mathbf{X}^n) = \arg \max_k p_{K, \mathbf{X}^n, T^n}(k, \mathbf{X}^n, T^n)$. We hereafter omit the subscripts of probability distribution. According to the marginalization in terms of Z^n , we have

$$\begin{aligned}p(k, \mathbf{X}^n, T^n) &= \sum_{z^n} p(k, z^n, \mathbf{X}^n, T^n) \\ &= \sum_{z^n} p(\mathbf{X}^n \mid z^n, T^n, k) p(z^n \mid T^n, k) p(T^n, k).\end{aligned}$$

Here, $p(\mathbf{X}^n \mid z^n, T^n, k) = p(\mathbf{X}^n \mid z^n)$, which follows from the fact that $\mathbf{X}^n$ and (T^n, K) are conditionally independent as Z^n d-separates $\mathbf{X}^n$ and (T^n, K) when Z^n is given [21]. As well, $p(T^n, k) = p(T^n)p(k)$, because T^n and K are conditionally independent because Z^n d-separates T^n and K when Z^n is not given. Since $\mathbf{X}^n$, Z^n and T^n consists of independent copies of $\mathbf{X}$, Z , and T , respectively, we have

$$\begin{aligned}p(k, \mathbf{X}^n, T^n) &= \sum_{z^n} p(\mathbf{X}^n \mid z^n) p(z^n \mid T^n, k) p(T^n) p(k) \\ &= p(k) \prod_{j=1}^n \sum_z p(\mathbf{X}_j \mid z_j) p(z \mid T_j, k) p(T_j).\end{aligned}$$

According to $Z^{(k)} = g(k, T)$ and Bayes' theorem, we have

$$\begin{aligned}p(k, \mathbf{X}^n, T^n) &= p_K(k) \prod_{j=1}^n p(\mathbf{X}_j \mid Z_j^{(k)}) p(T_j) \\ &= p(k) \prod_{j=1}^n \frac{p(Z_j^{(k)} \mid \mathbf{X}_j) p(\mathbf{X}_j) p(T_j)}{p(Z_j^{(k)})}.\end{aligned}\quad (3)$$

Taking the logarithm of Equation (3), we have

$$\begin{aligned}\log p(k, \mathbf{X}^n, T^n) &= \log p(k) + \sum_{j=1}^n \log p(\mathbf{X}_j) p(T_j) \\ &\quad + \sum_{j=1}^n \log p(Z_j^{(k)} \mid \mathbf{X}_j) - \sum_{j=1}^n \log p(Z_j^{(k)}).\end{aligned}\quad (4)$$

Consider the arg max of Equation (4) in terms of k . The first term $\log p_K(k)$ has no impact on the arg max because K is uniformly distributed. The second term $\sum_j \log p(\mathbf{X}_j) p(T_j)$ also has no impact because it is independent of k . Thus, assuming that Z is uniformly distributed, we conclude

$$\begin{aligned}D(T^n, \mathbf{X}^n) &= \arg \max_k \log p(k \mid \mathbf{X}^n, T^n) \\ &= \arg \max_k \sum_{j=1}^n \log p_{Z \mid \mathbf{X}}(Z_j^{(k)} \mid \mathbf{X}_j),\end{aligned}\quad (5)$$

is an optimal distinguisher, as required.

The second equality is proven in a similar manner as [77, Equation (2)]. Marginalizing $A_0, A_1, \dots, A_d$, we have

$$p(z \mid \mathbf{X}) = \sum_{a_0, a_1, \dots, a_d} p(z, a_0, a_1, \dots, a_d \mid \mathbf{L}_0, \mathbf{L}_1, \dots, \mathbf{L}_d).$$

Here, Z is conditionally independent of $\mathbf{L}^d$ if A^d is given, which d-separates Z and $\mathbf{L}^d$. Because $A_0, A_1, \dots, A_d$ are mutually independent when Z is not given ($\mathbf{L}_0, \mathbf{L}_1, \dots, \mathbf{L}_d$ are same) and $z = a_0 \oplus a_1 \oplus \dots \oplus a_d$, it holds that

$$\begin{aligned}p(z \mid \mathbf{X}) &= \sum_{a_0, a_1, \dots, a_d} p(z \mid a_0, a_1, \dots, a_d) \prod_i p(a_i \mid \mathbf{L}_i) \\ &= \sum_{z=a_0 \oplus a_1 \oplus \dots \oplus a_d} \prod_{i=0}^d p(a_i \mid \mathbf{L}_i).\end{aligned}\quad (6)$$

By substituting Equation (6) to Equation (5), we complete the proof. $\square$